

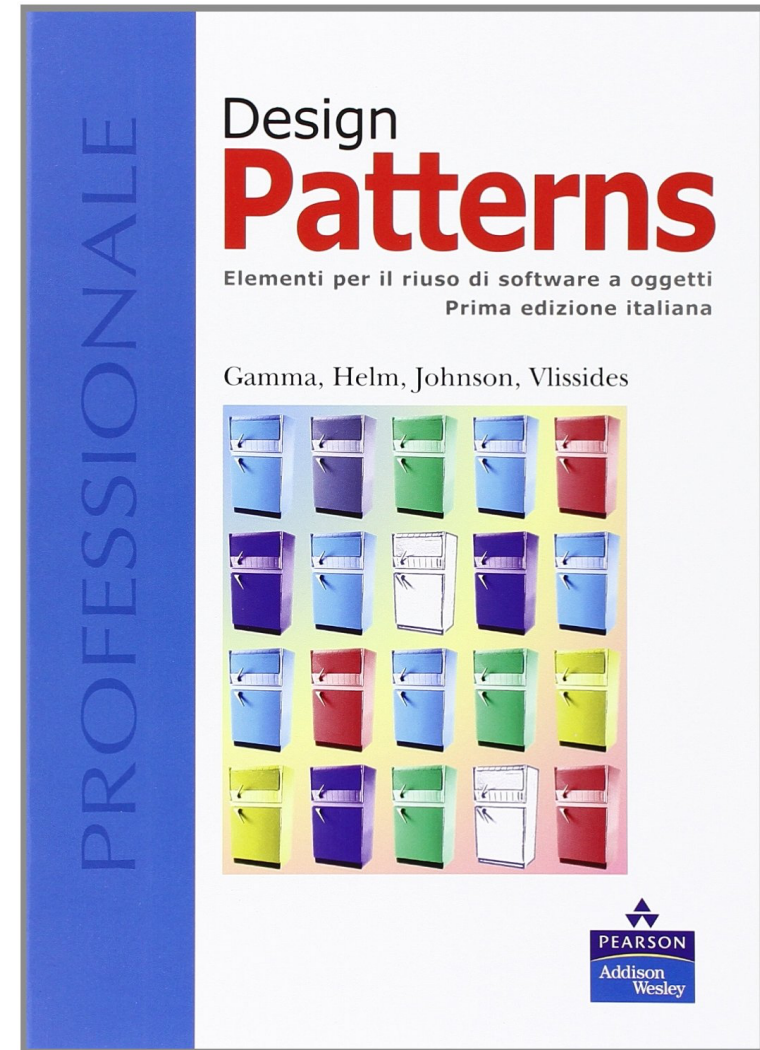
Analisi e Progettazione del Software

Applicare design pattern GoF

Oliviero Riganelli

Design Pattern

- I design pattern sono una soluzione progettuale comune a un problema di progettazione ricorrente
- I 23 design pattern GoF sono classificati in base al loro scopo (creazionale, strutturale o comportamentale)
- La soluzione è rappresentata da un insieme di oggetti e classi a cui sono date delle responsabilità e che collaborano per risolvere il problema di interesse



Design Pattern GoF

		Scopo		
		<i>Crazionale</i>	<i>Strutturale</i>	<i>Comportamentale</i>
Raggio d'azione	<i>Classi</i>	Factory Method	Adapter	Interpreter Template Method
	<i>Oggetti</i>	Abstract Factory Builder Prototype Singleton	Adapter (object) Bridge Composite Decorator Facade Flyweight Proxy	Chain of responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

Adapter

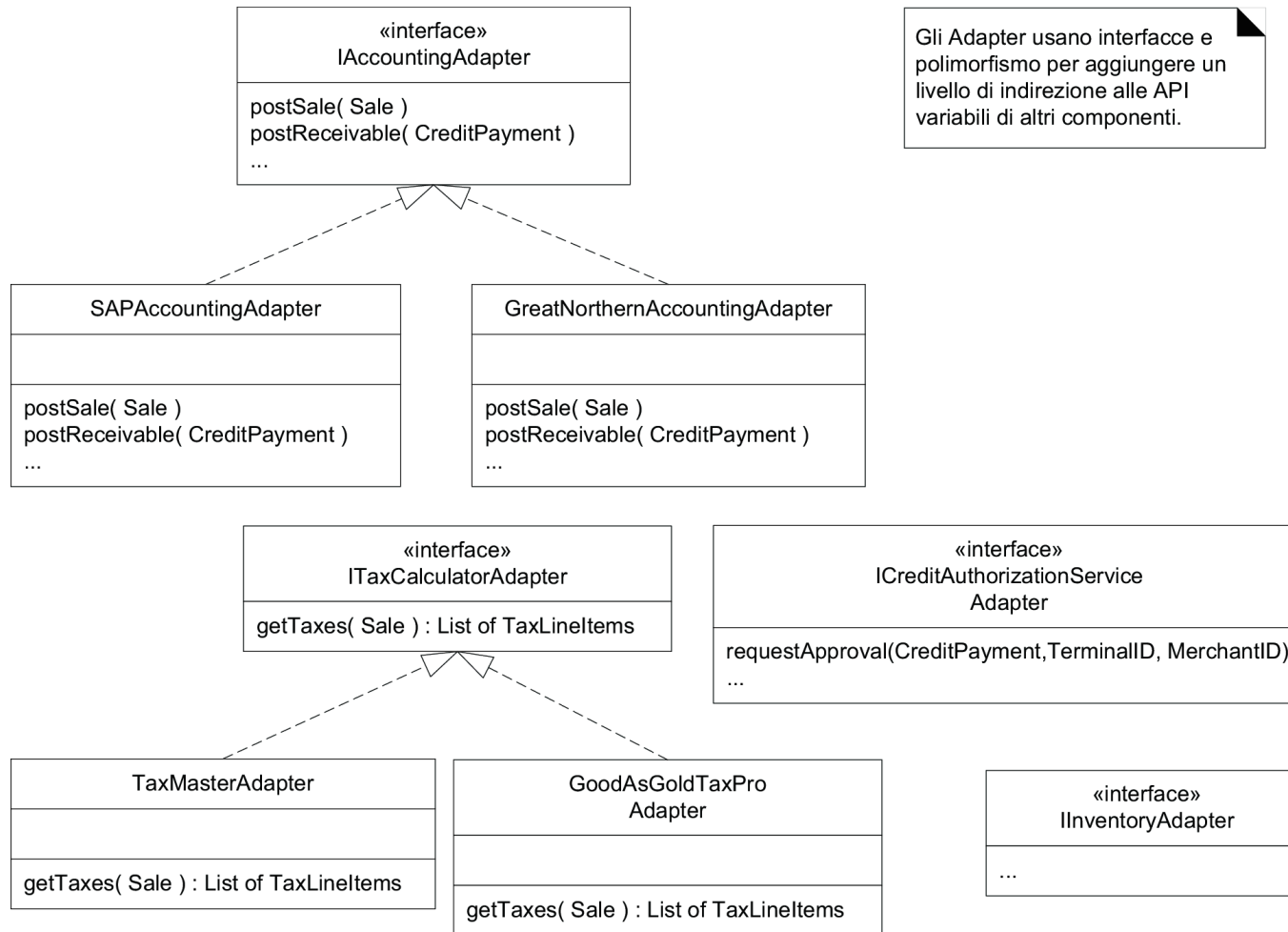
Nome:	Adapter
Problema:	Come gestire interfacce incompatibili, o fornire un'interfaccia stabile a componenti simili ma con interfacce diverse?
Soluzione: (consiglio)	Converti l'interfaccia originale di un componente in un'altra interfaccia, attraverso un oggetto adattatore intermedio.

- **Interfacce incompatibili:** si consideri una coppia di oggetti software in una relazione client server. Si parla di interfacce incompatibili quando l'oggetto server offre servizi di interesse per l'oggetto client, ma l'oggetto client vuole fruire di questi servizi in una modalità diversa da quella prevista dall'oggetto server
- **Componenti simili con interfacce diverse:** Ci sono più oggetti server che offrono servizi simili; questi oggetti hanno interfacce simili ma diverse tra loro. Un oggetto client vuole fruire dei servizi offerti da uno tra questi oggetti server

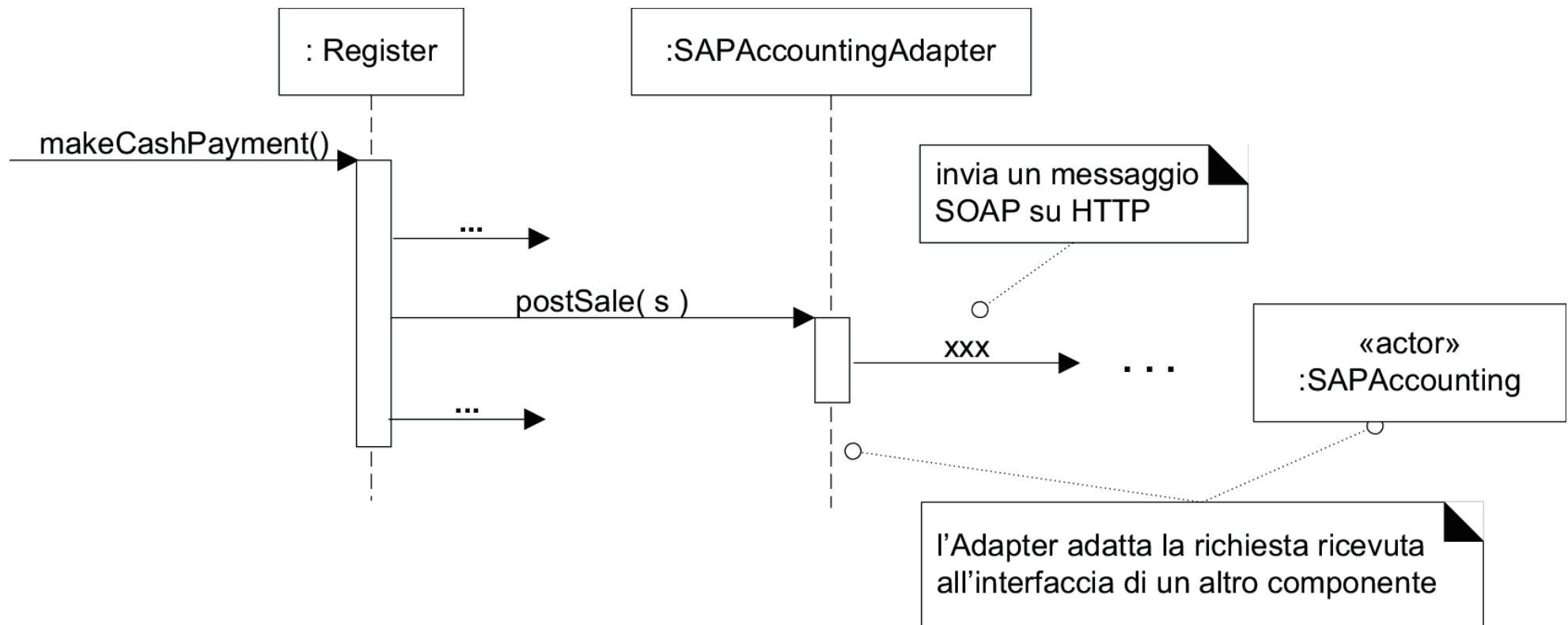
Esempio

- Il sistema POS NextGen deve supportare diversi tipi di servizi esterni prodotti da terze parti, compresi i sistemi per la contabilità e per l'inventario, i servizi di autorizzazione ai pagamenti, i servizi per il calcolo delle imposte, tra gli altri.
- Ciascuno di essi ha una propria api, diversa da quelle degli altri, che non può essere modificata

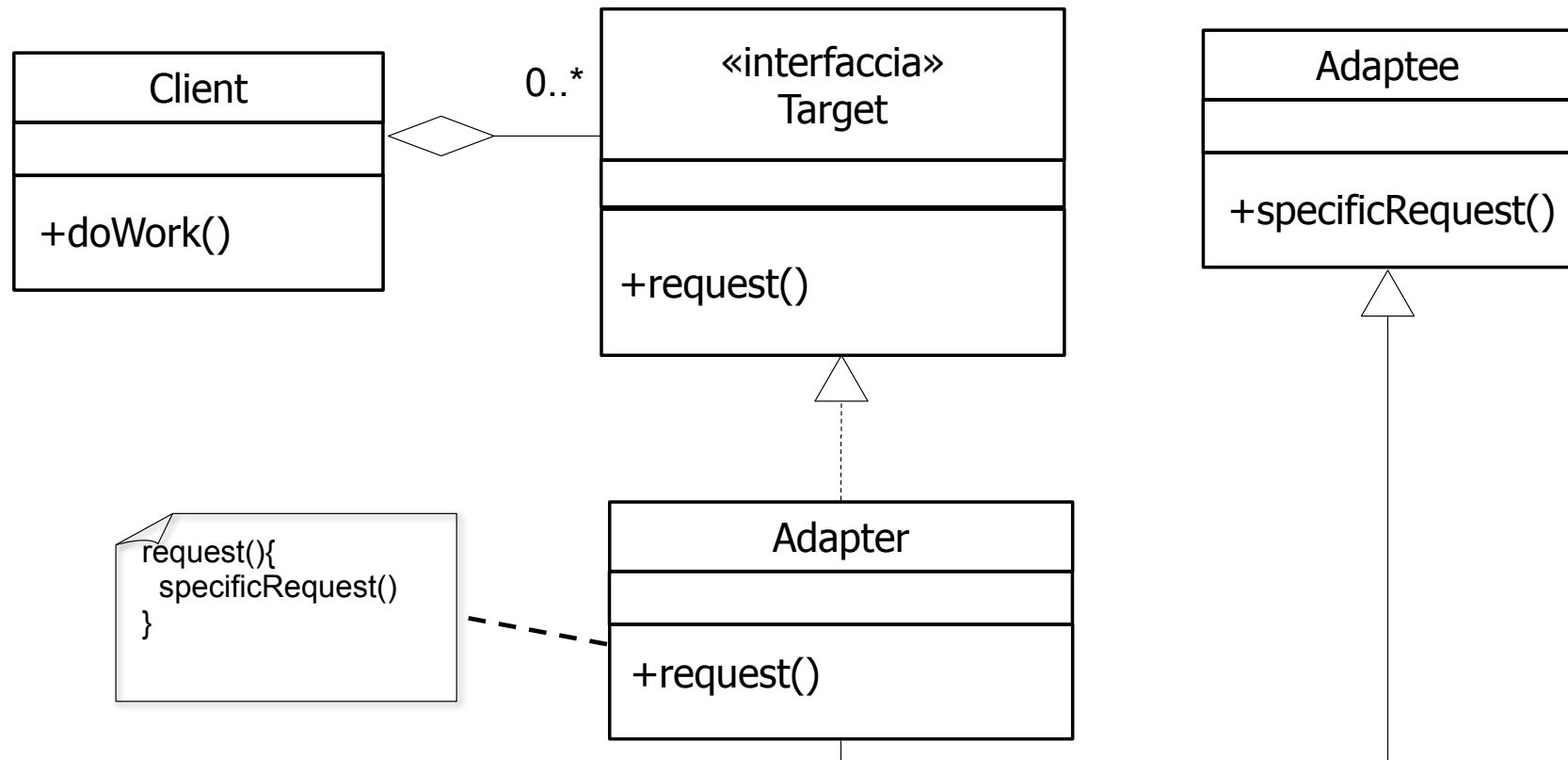
Soluzione



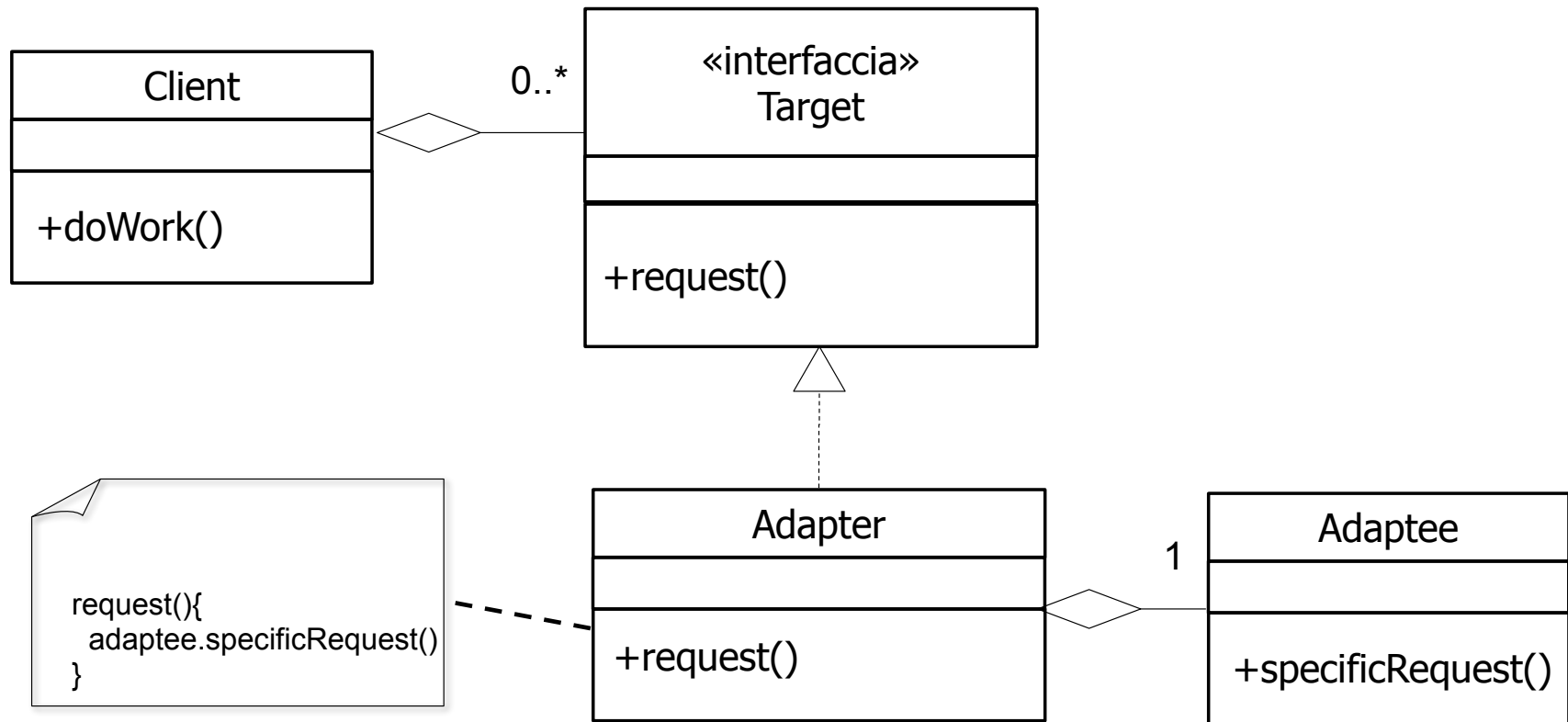
Soluzione



Adapter (Classe) Struttura



Adapter (Oggetto) Struttura



I pattern GRASP come generalizzazione di altri pattern

Caratterizzazione del design pattern Adapter in termini di pattern GRASP

- sostiene **Protected Variations** rispetto al cambiamento di interfacce esterne o di package di terze parti
- attraverso l'uso di un oggetto **Indirection**
- che applica interfacce e **Polymorphism**

Molti design pattern possono essere analizzati in termini dei pattern GRASP

- i pattern GRASP possono semplificare la comprensione dei molti design pattern esistenti
- per un progettista è utile saper vedere i principi di base dei numerosi design pattern esistenti

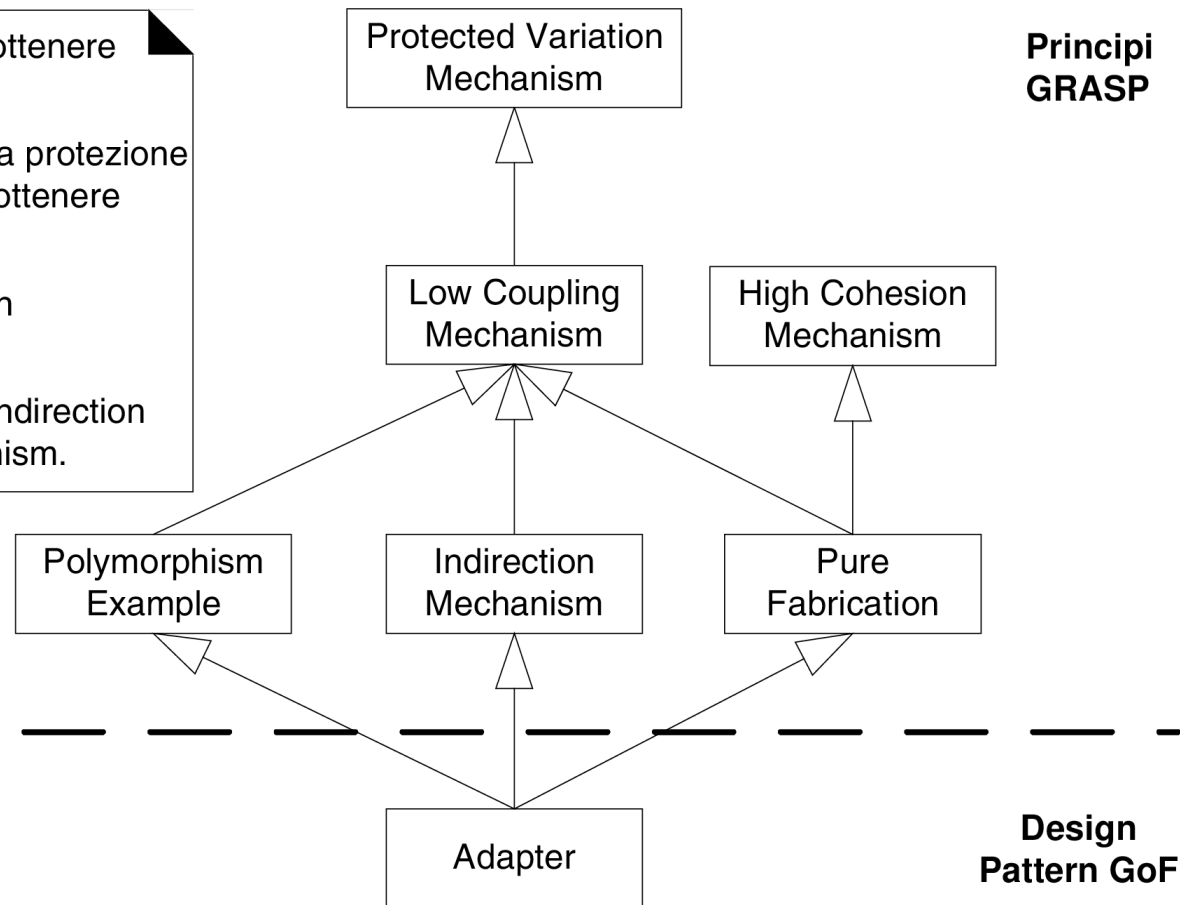
Adapter e GRASP

Un accoppiamento basso è un modo per ottenere una protezione in un punto di variazione.

Il polimorfismo è un modo per ottenere una protezione in un punto di variazione, e un modo per ottenere un accoppiamento basso.

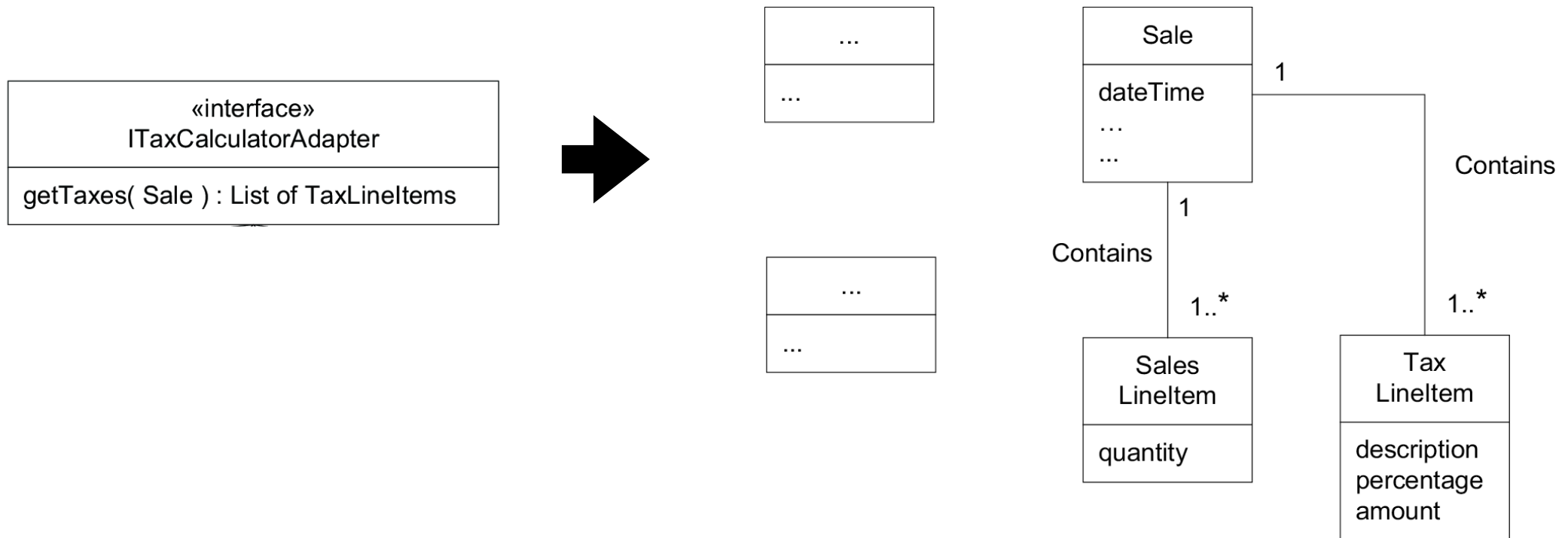
Una indirezione è un modo per ottenere un accoppiamento basso.

Il design pattern Adapter è una specie di Indirection e Pure Fabrication, che utilizza Polymorphism.



Scoperte di analisi durante la progettazione

- L'operazione `getTaxes` di `ITaxCalculatorAdapter` restituisce una lista di `TaxLineItem` - un nuovo concetto del dominio



Factory

L'uso del pattern Adapter pone i seguenti problemi

- chi crea gli oggetti adattatori?
- come stabilire quale classe di adattatore creare?

La responsabilità di creare gli adattatori (che non sono oggetti del dominio) non va assegnata a oggetti del dominio

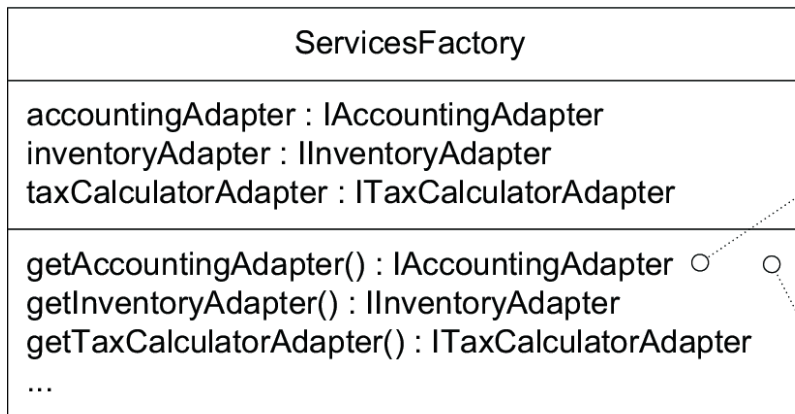
- oltre la logica applicativa pura
- diminuisce la coesione
- non sostiene l'obiettivo della progettazione per una **separazione degli interessi**
- va piuttosto usato un oggetto Pure Fabrication

Factory

Nome:	Factory
Problema:	Chi deve essere responsabile della creazione di oggetti quando ci sono delle considerazioni speciali, come una logica di creazione complessa, quando si desidera separare le responsabilità di creazione per una coesione migliore, e così via?
Soluzione: (consiglio)	Crea un oggetto Pure Fabrication chiamato una Factory che gestisce la creazione.

- Separa la responsabilità delle creazioni complesse in oggetti di supporto coesi
- Nasconde la logica di creazione potenzialmente complessa.
- Consente l'introduzione di strategie per la gestione delle risorse come caching o riciclaggio degli oggetti

Esempio POS NextGen



Si noti che i metodi factory restituiscono degli oggetti di cui il tipo è un'interfaccia, anziché una classe, in modo tale che la factory possa restituire una qualsiasi implementazione dell'interfaccia.

```
if ( accountingAdapter == null ) {  
  
    // un approccio basato sulla riflessione o data-driven per trovare la classe giusta:  
    // leggerla da una proprietà esterna  
  
    String className = System.getProperty( "accounting.adapter.class.name" );  
    accountingAdapter = (IAccountingAdapter) Class.forName( className ).newInstance();  
  
}  
return accountingAdapter;
```

File properties

```
# Properties per POS NextGen
accounting.adapter.class.name=
com.foo.nextgen.services.accounting.SAPAccountingAdapter
...
```

```
/* lettura di un file di properties
* ad es., nel caso d'uso d'avviamento */
Properties properties = new Properties();
try {
    properties.load(new FileInputStream("POS.properties"));
} catch (IOException e) {}
```

```
/* accesso a properties */
String className =
properties.getProperty("accounting.adapter.class.name");
```


Discussione e Pattern Correlati

Discussione

- nell'esempio, la logica della creazione è data-driven – sostiene **Protected Variations** rispetto a cambiamenti nella scelta dell'adattatore da utilizzare

Pattern Correlati

- spesso si accede agli oggetti Factory mediante il pattern **Singleton**

Singleton

La classe `ServicesFactory` pone i seguenti problemi

- chi crea l'oggetto `Factory`? di solito è necessaria una sola istanza della `Factory`
- come può essere acceduto l'oggetto `Factory`? deve essere visibile in diversi punti del codice, dove serve l'accesso ai servizi esterni

Singleton

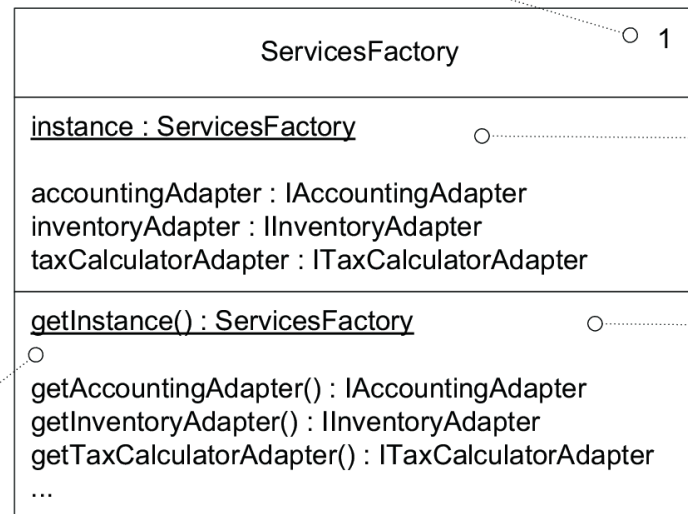
Nome:	Singleton
Problema:	È consentita (o richiesta) esattamente una sola istanza di una classe, ovvero un “singleton”. Gli altri oggetti hanno bisogno di un punto di accesso globale e singolo a questo oggetto.
Soluzione: (consiglio)	Definisci un metodo statico (di classe) della classe che restituisce l'oggetto singleton.

- assicurare che una classe abbia una sola istanza e fornire un punto d'accesso globale a tale istanza

Esempio

notazione UML: questo '1' può essere usato, opzionalmente, per indicare che sarà creata una sola istanza (un singleton)

notazione UML: nel rettangolo di una classe, un attributo o un metodo sottolineato indica un membro statico (di classe), anziché un membro d'istanza



attributo statico
singleton

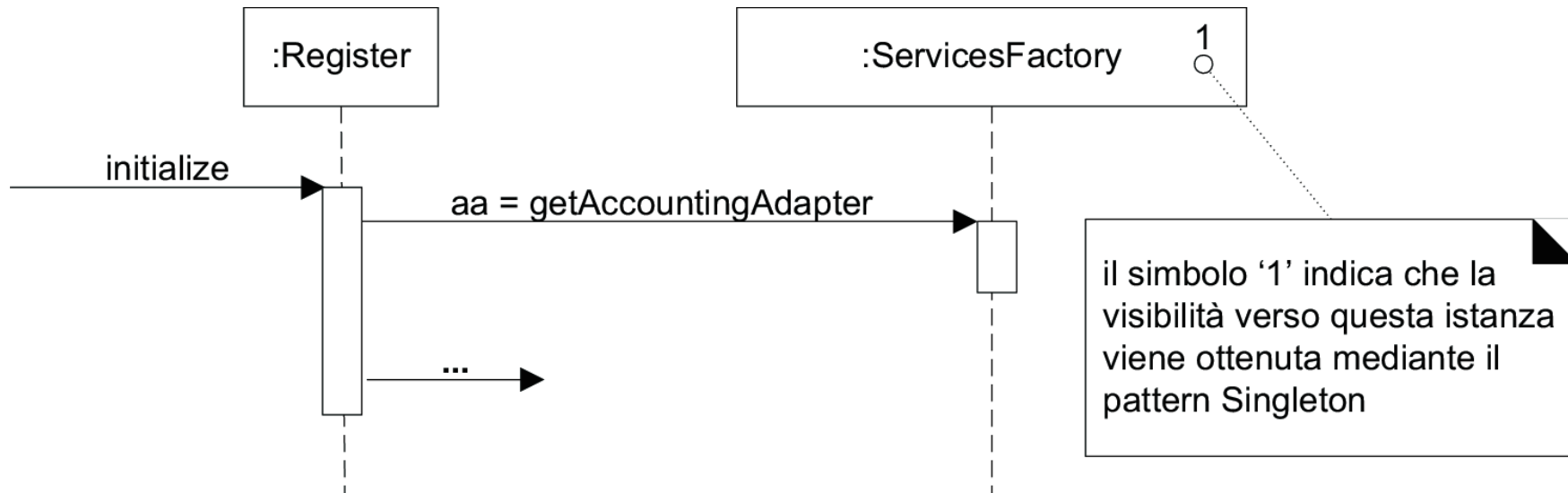
metodo statico
singleton

```
// metodo statico
public static synchronized ServicesFactory getInstance() {
    if ( instance == null ) {
        instance = new ServicesFactory();
    }
    return instance;
}
```

Esempio

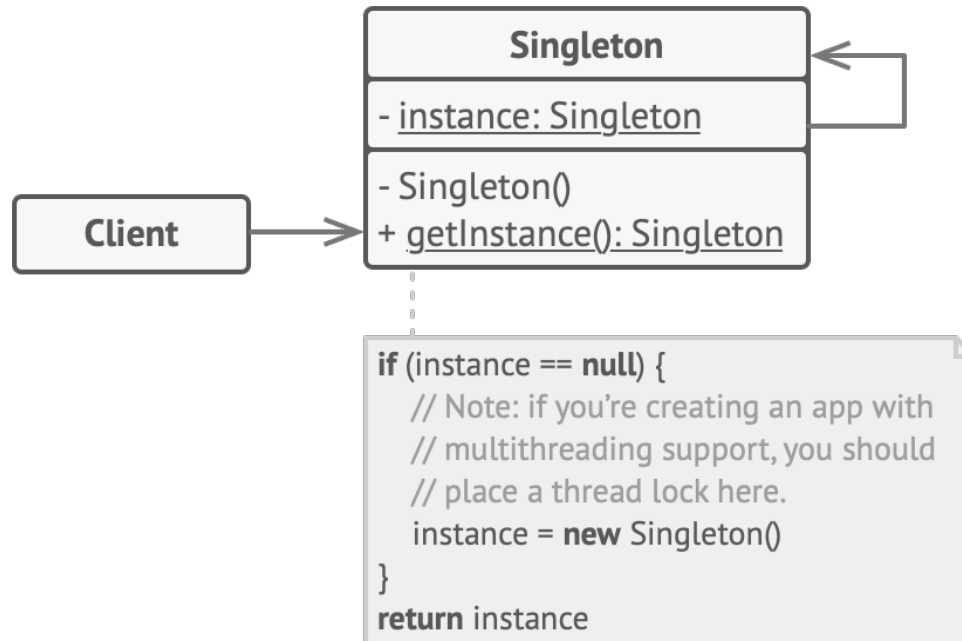
```
public class Register {  
  
    public void initialize() {  
        ... do some work ...  
  
        // accesso alla Factory singleton attraverso  
        // la chiamata a getInstance  
  
        accountingAdapter =  
            ServicesFactory.getInstance().getAccountingAdapter();  
  
        ... do some work ...  
    }  
    // altri metodi...  
} // fine della classe
```

Esempio



```
accountingAdapter =  
    ServicesFactory.getInstance().getAccountingAdapter();
```

Struttura



Tipiche implementazioni

```
public class ServicesFactory {  
    private static ServicesFactory  
        _instance = new ServicesFactory();  
  
    private ServicesFactory() {  
        //construct object ...  
    }  
  
    public static ServicesFactory getInstance()  
    {  
        return _instance;  
    }  
  
    ...  
  
    Perché il costruttore privato?  
    Cosa c'è di errato nella  
    seconda implementazione?
```

```
public class ServicesFactory {  
    private static ServicesFactory _instance;  
  
    private ServicesFactory() {  
        // construct object ...  
    }  
  
    public static ServicesFactory getInstance() {  
        if (_instance==null) {  
            _instance = new ServicesFactory();  
        }  
        return _instance;  
    }  
  
    ...  
  
    Perché il costruttore privato?  
    Cosa c'è di errato nella  
    seconda implementazione?
```


Tipiche implementazioni

```
public class ServicesFactory {  
    private static ServicesFactory  
        _instance = new ServicesFactory();  
  
    private ServicesFactory() {  
        //construct object ...  
    }  
}
```

```
public static ServicesFactory getInstance()  
{  
    return _instance;  
}
```

...

Perché il costruttore privato?
Cosa c'è di errato nella
seconda implementazione?

```
public class ServicesFactory {  
    private static ServicesFactory _instance;  
  
    private ServicesFactory() {  
        // construct object ...  
    }  
}
```

```
public static synchronized ServicesFactory getInstance()  
    if (_instance==null) {  
        _instance = new ServicesFactory();  
    }  
    return _instance;  
}
```

...

Vantaggi

- Singleton definisce un punto di accesso globale a un unico oggetto istanza di una classe, consentendo l'inizializzazione pigra di questo oggetto
 - *questa soluzione è spesso preferibile all'uso di variabili globali (statiche)*

Controindicazioni

- Singleton definisce oggetti con una visibilità globale
 - *la visibilità globale va spesso evitata perché, se abusata, ha come conseguenza un **aumento dell'accoppiamento** complessivo*
- può essere non efficace nella realizzazione di applicazioni distribuite
 - *poiché ciascuna VM potrebbe avere un'istanza diversa della classe Singleton*

Pattern Correlati

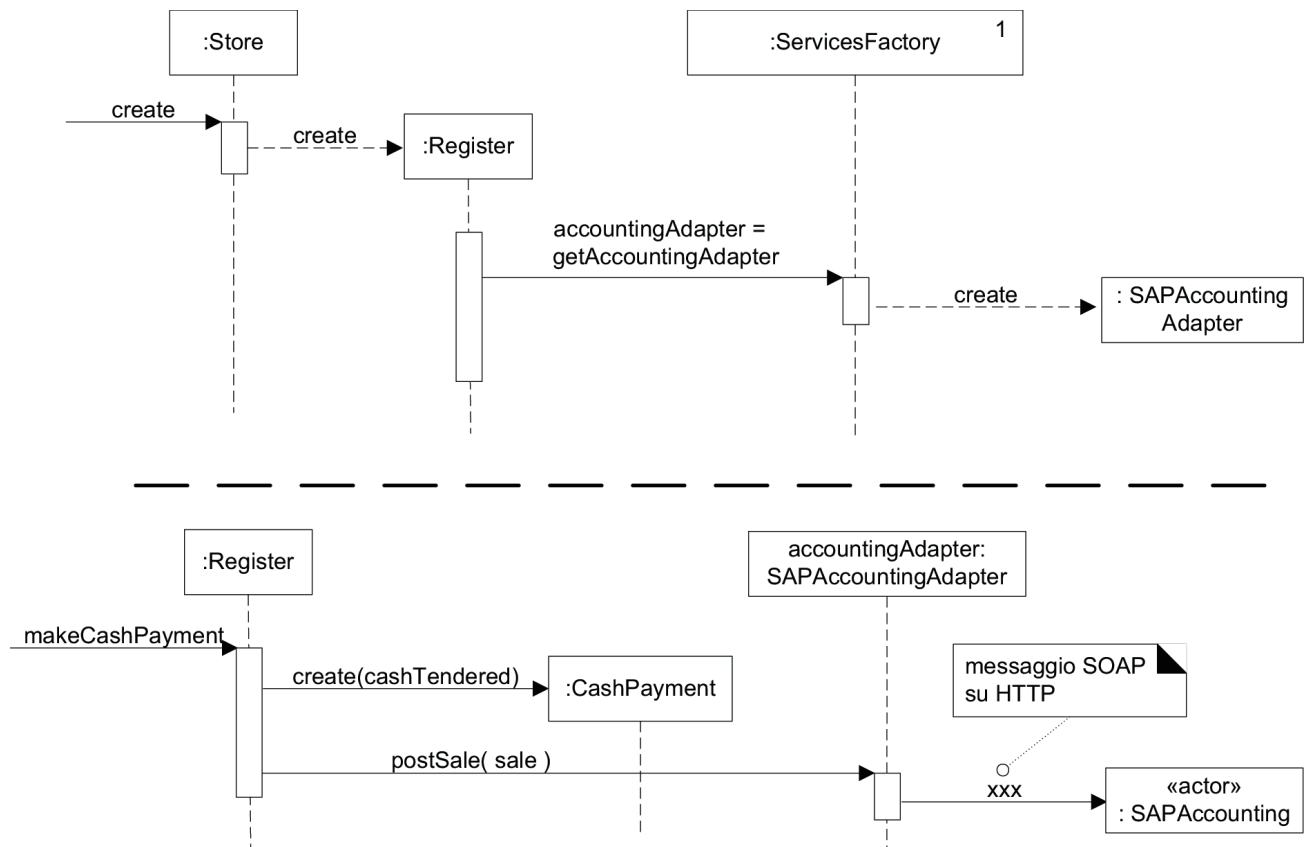
- il pattern Singleton viene spesso usato per oggetti **Factory** e **Facade**

Conclusioni sul problema dei servizi esterni con interfacce variabili

- È possibile realizzare Protected Variations rispetto a interfacce variabili di sistemi esterni usando una combinazione dei design pattern Adapter, Factory e Singleton
- possibile creare progetti usando un insieme di principi o design pattern come elementi costruttivi di base
- approccio metodico alla creazione e alla spiegazione di progetti

Conclusioni sul problema dei servizi esterni con interfacce variabili

- “Per gestire il problema delle interfacce variabili per i servizi esterni, usiamo alcuni Adapter generati da una Factory Singleton”



Strategy

Il prossimo problema di progettazione da risolvere è fornire una logica più complessa per la determinazione del prezzo scontato di una vendita

- Durante un periodo può essere pari al 10% di sconto su tutte le vendite
- Successivamente può essere uno sconto 10 € se il totale di vendita è superiore a 200 €
- E un'infinità di altre varianti

Come è possibile progettare per questi algoritmi per il calcolo del prezzo scontato e per la loro variabilità

Strategy

Nome:	Strategy
Problema:	Come progettare per gestire un insieme di algoritmi o politiche variabili ma correlati? Come progettare per consentire di modificare questi algoritmi o politiche?
Soluzione: (consiglio)	Definisci ciascun algoritmo/politica/strategia in una classe separata, con un'interfaccia comune.

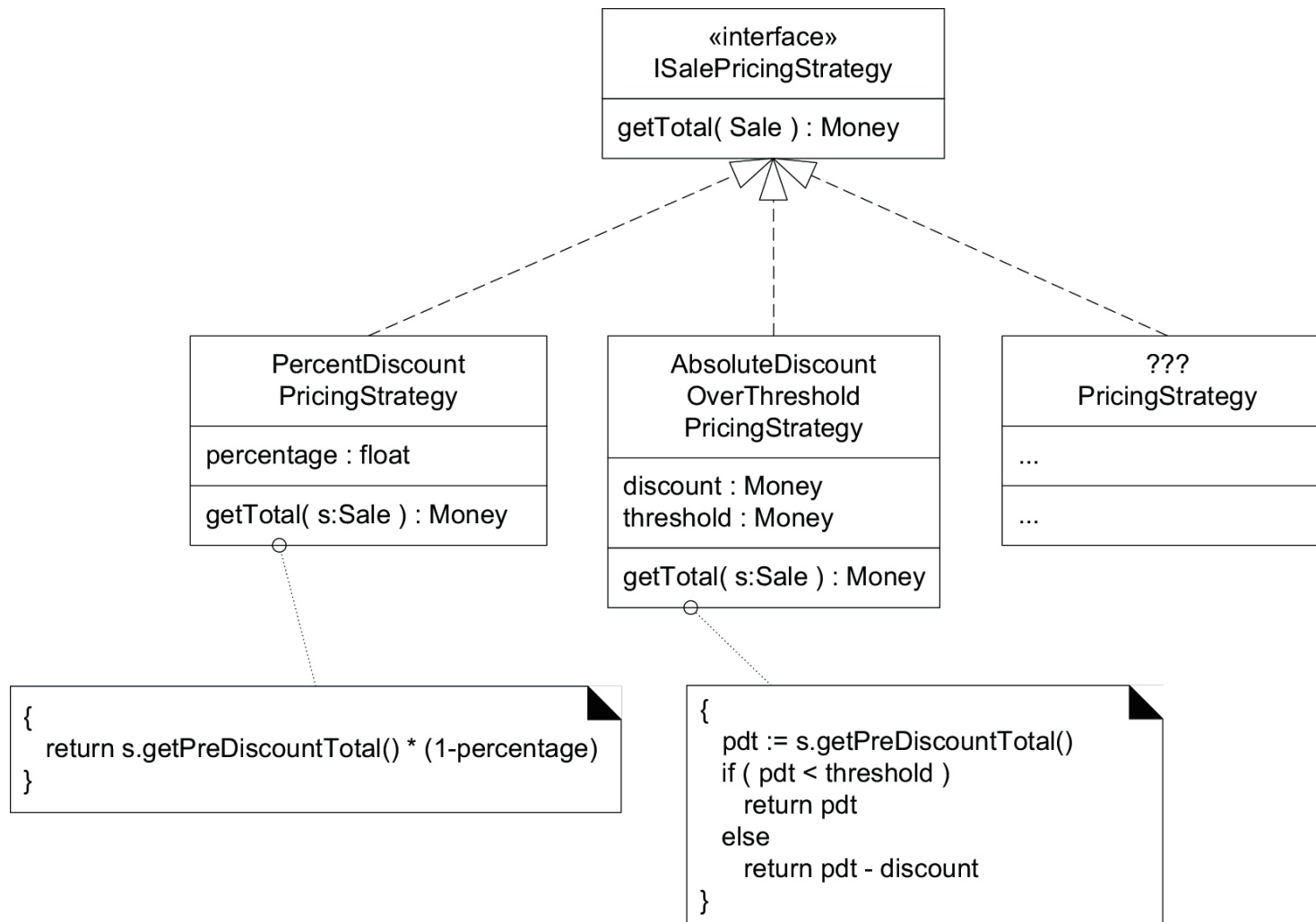
- Gli algoritmi di cui si parla nel pattern strategy sono correlati nel senso che risolvono tutti uno stesso problema

Strategy

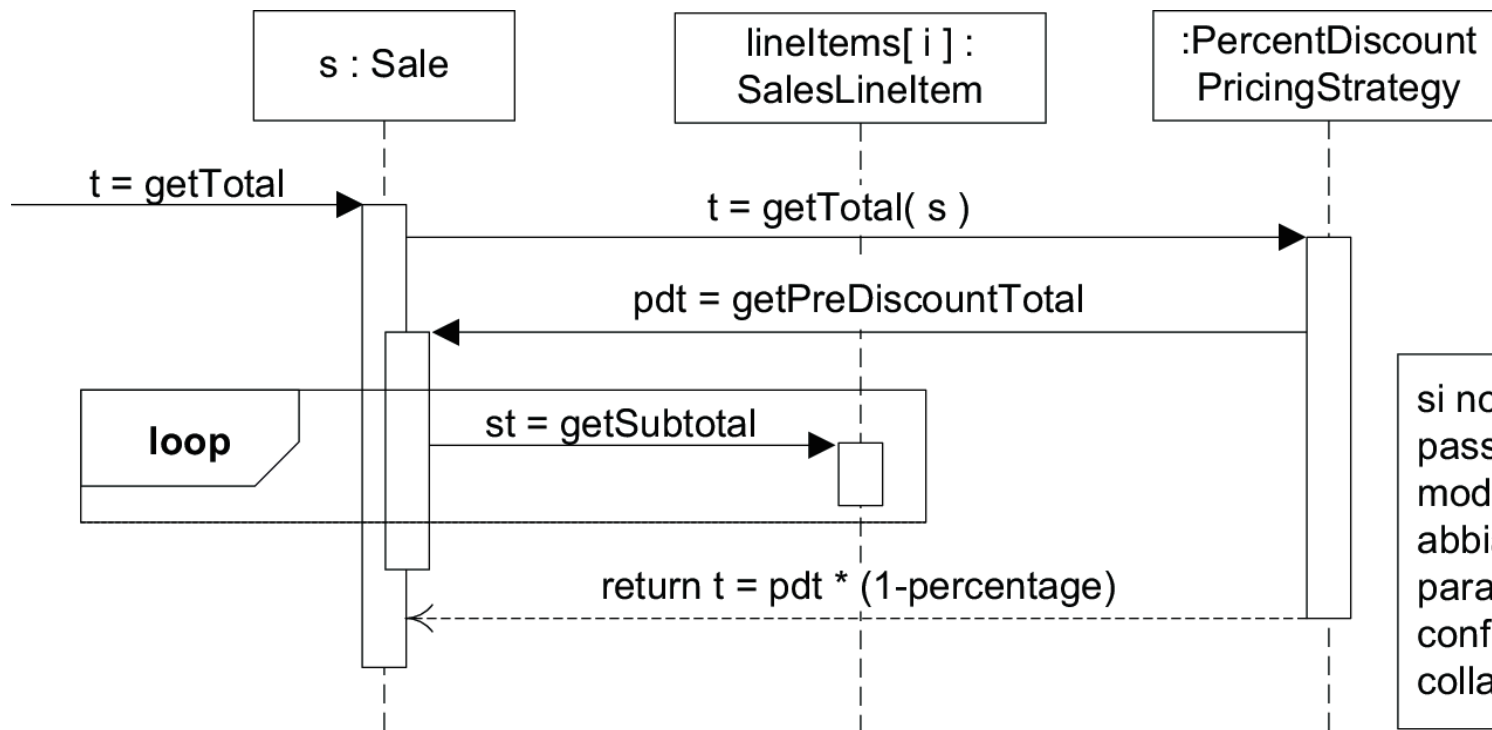
Nome:	Strategy
Problema:	Come progettare per gestire un insieme di algoritmi o politiche variabili ma correlati? Come progettare per consentire di modificare questi algoritmi o politiche?
Soluzione: (consiglio)	Definisci ciascun algoritmo/politica/strategia in una classe separata, con un'interfaccia comune.

- Gli algoritmi di cui si parla nel pattern strategy sono correlati nel senso che risolvono tutti uno stesso problema

Classi Strategy per la determinazione del prezzo

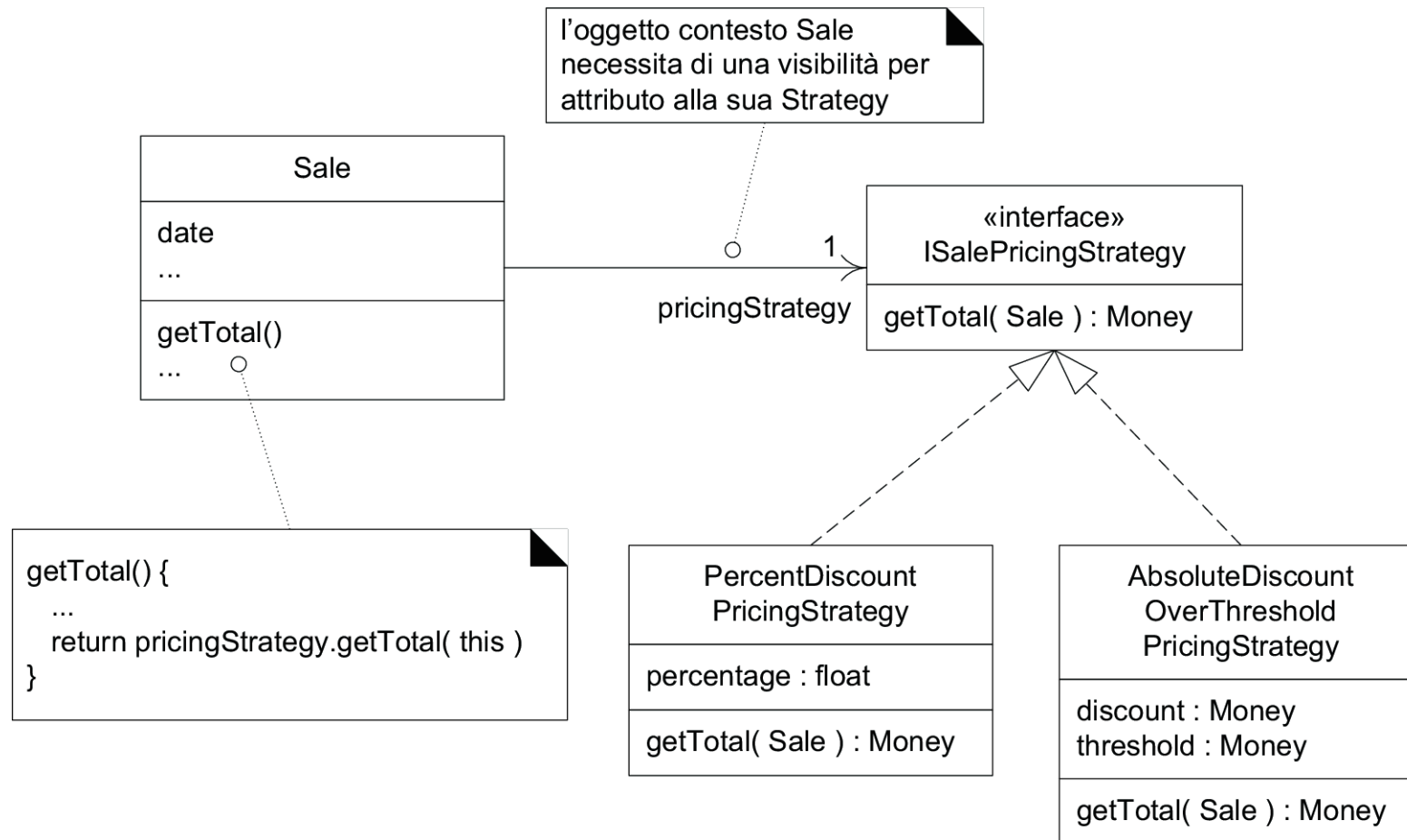


Collaborazione con Strategy



si noti che la Sale viene passata alla Strategy in modo che quest'ultima abbia una visibilità per parametro nei suoi confronti per ulteriori collaborazioni

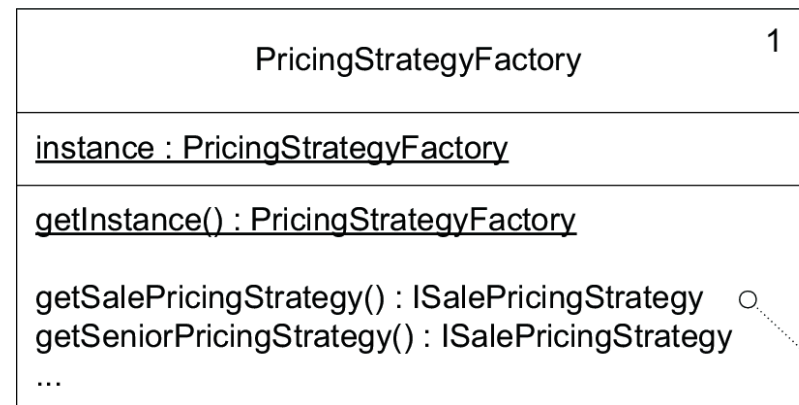
L'oggetto contesto necessita della visibilità per attributo nei confronti della sua strategia



Creare una Strategy con una Factory

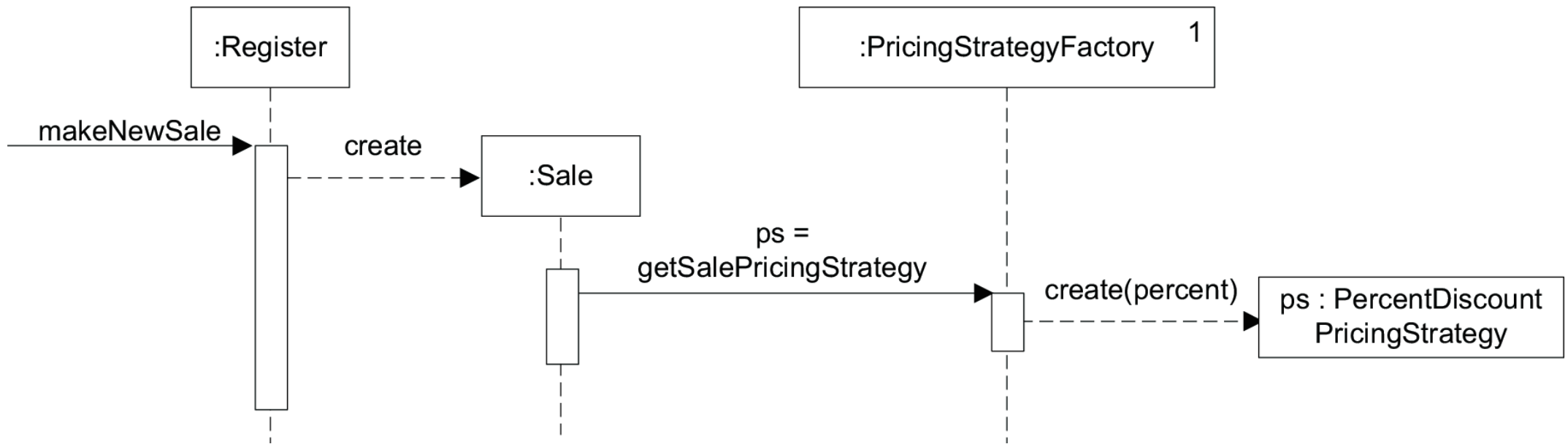
Ci sono diversi algoritmi o strategie di determinazione del prezzo, che cambiano nel corso del tempo

- Chi deve creare la strategia?

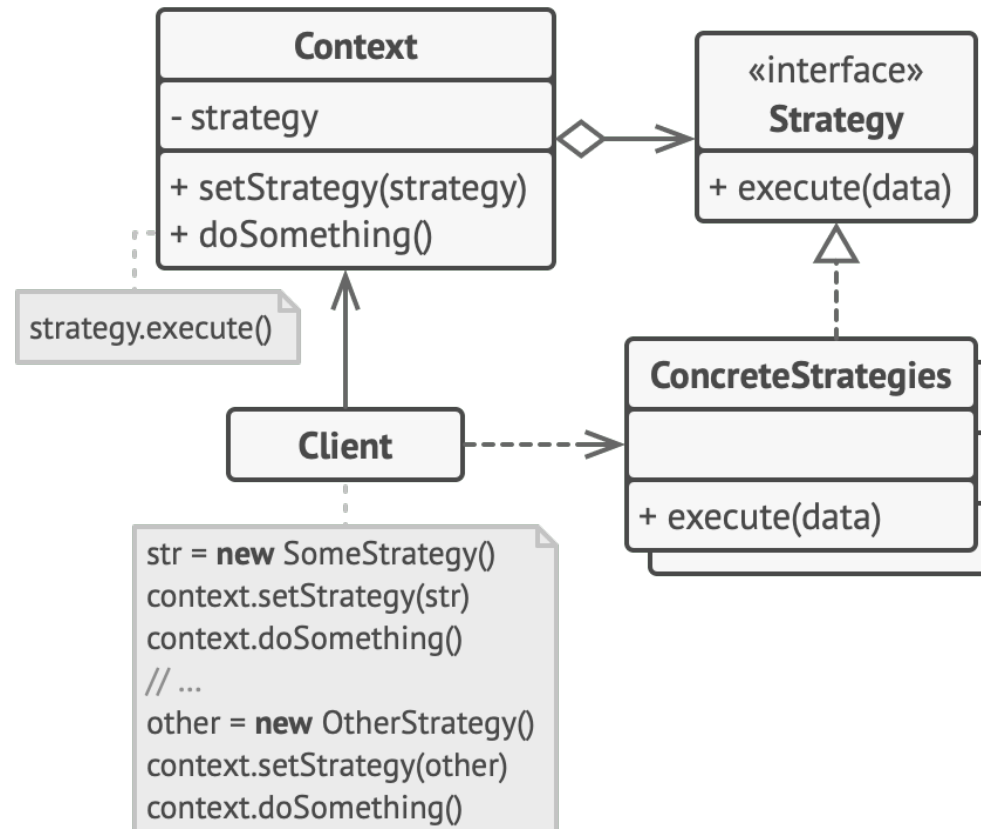


```
{
    String className = System.getProperty( "salepricingstrategy.class.name" );
    strategy = (ISalePricingStrategy) Class.forName( className ).newInstance();
    return strategy;
}
```

Creazione di una strategia



Struttura



Riepilogo e discussione

- L'applicazione del pattern **Strategy** e **Factory** consente di ottenere **Protected Variations** rispetto alle politiche di determinazione del totale scontato delle vendite
- Strategy si basa su **Polymorphism** e sulle interfacce per consentire degli algoritmi inseribili in un progetto

Pattern collegati

- Strategy si basa su **Polymorphism** e fornisce **Protected Variations** rispetto agli algoritmi variabili
- Le strategie sono spesso create da una **Factory**

Composite

Come gestire il caso di politiche di determinazione del prezzo multiple e conflittuali? Si supponga che un negozio abbia le seguenti politiche in vigore oggi (Giovedì)

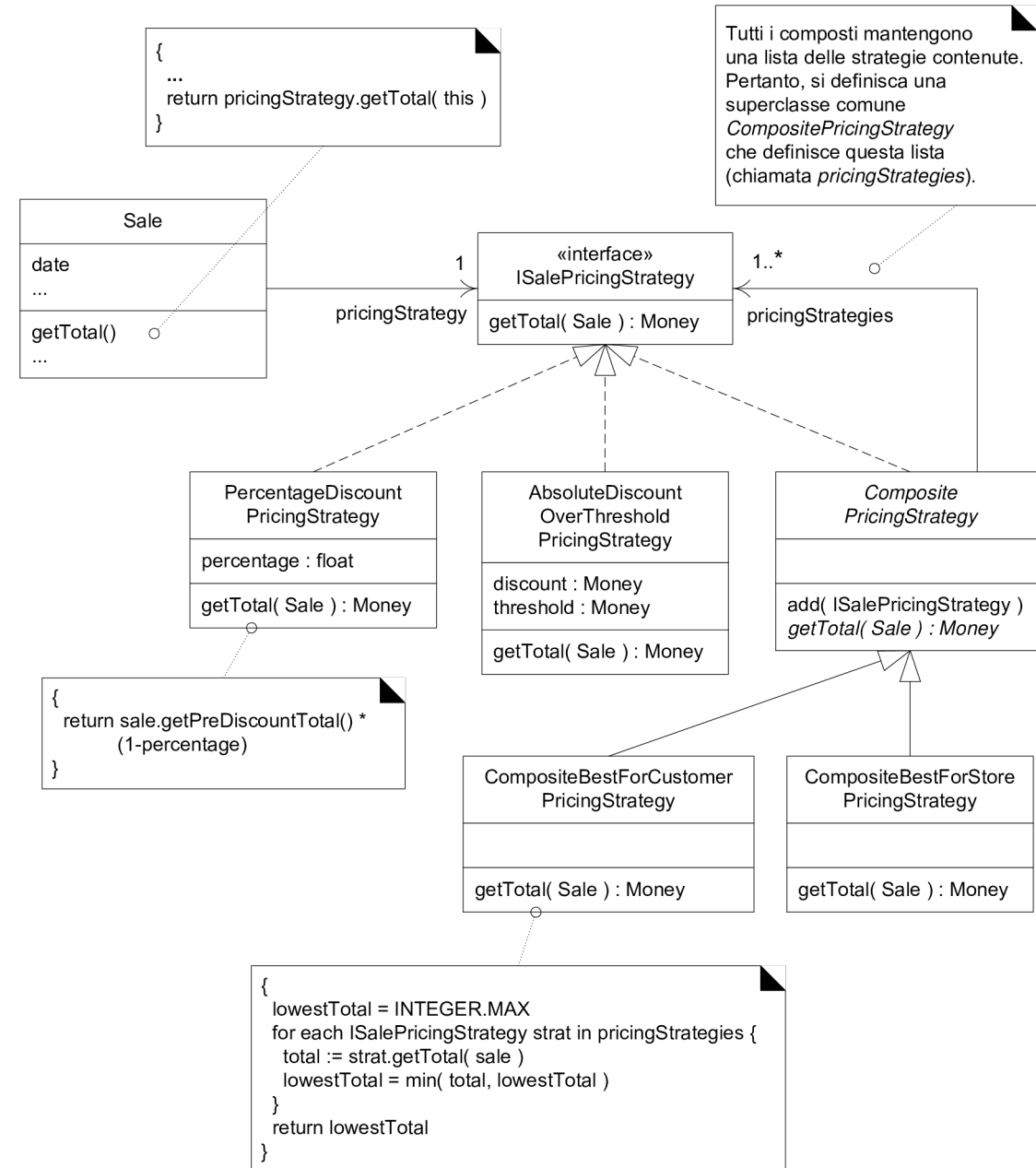
- Politica di sconto del 20% agli anziani
- Sconto del 15% ai clienti abituali per vendite superiori a 400 €
- Il Giovedì viene applicato uno sconto di 50€ sugli acquisti superiori a 500€
- Chi compra una confezione di tè riceve uno sconto del 15% su ogni cosa

Composite

C'è un modo per modificare il progetto in modo tale che l'oggetto Sale non sappia se si trova di fronte a una sola o più strategie di determinazione del prezzo e offrire anche un progetto per la risoluzione dei conflitti?

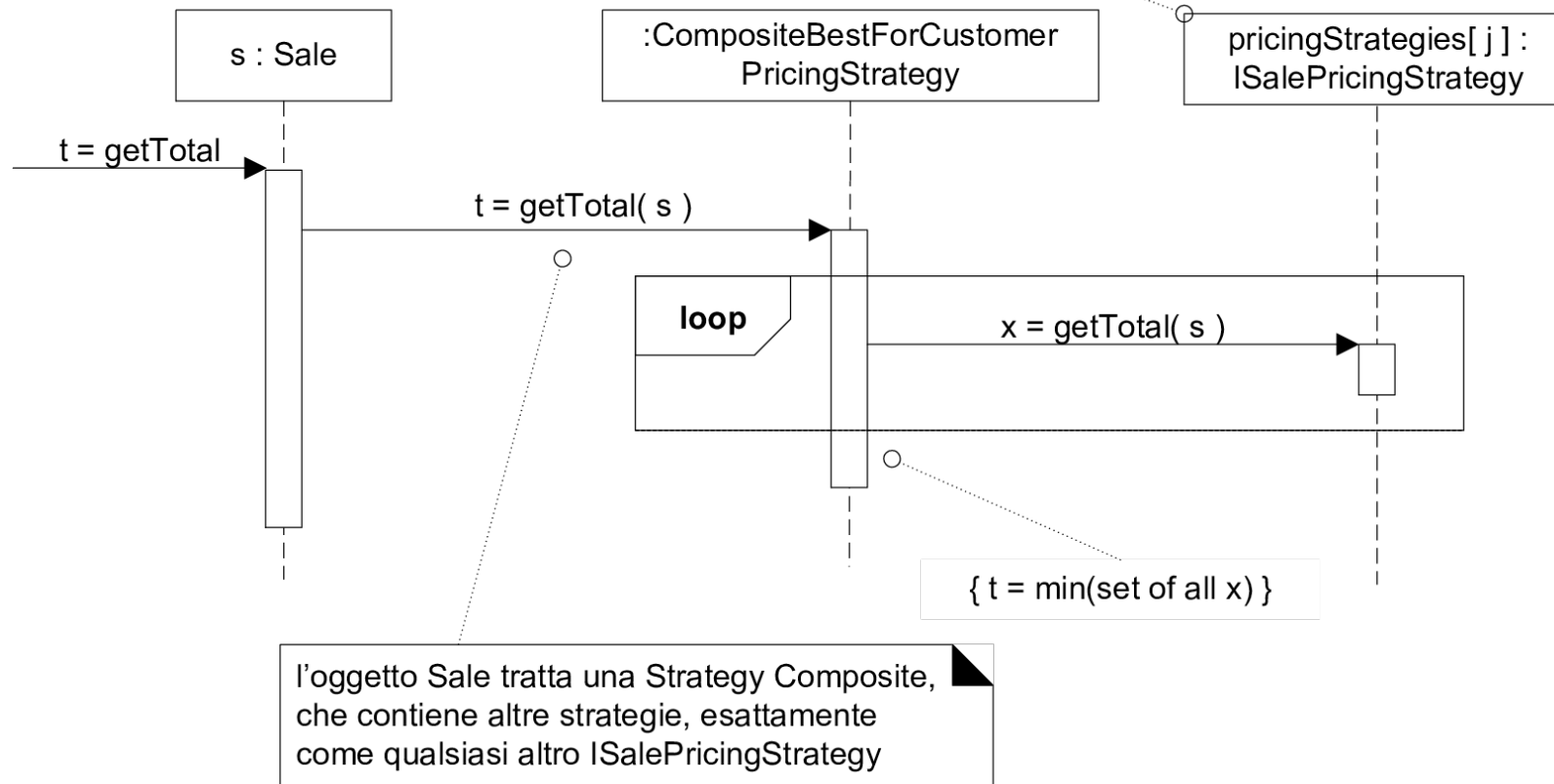
Nome:	Composite
Problema:	Come trattare un gruppo o una struttura composta di oggetti (polimorficamente) dello stesso tipo nello stesso modo di un oggetto non composto (atomico)?
Soluzione: (consiglio)	Definisci le classi per gli oggetti composti e atomici in modo che implementino la stessa interfaccia.

Applicazione del pattern Composite



Collaborazione con un Composite

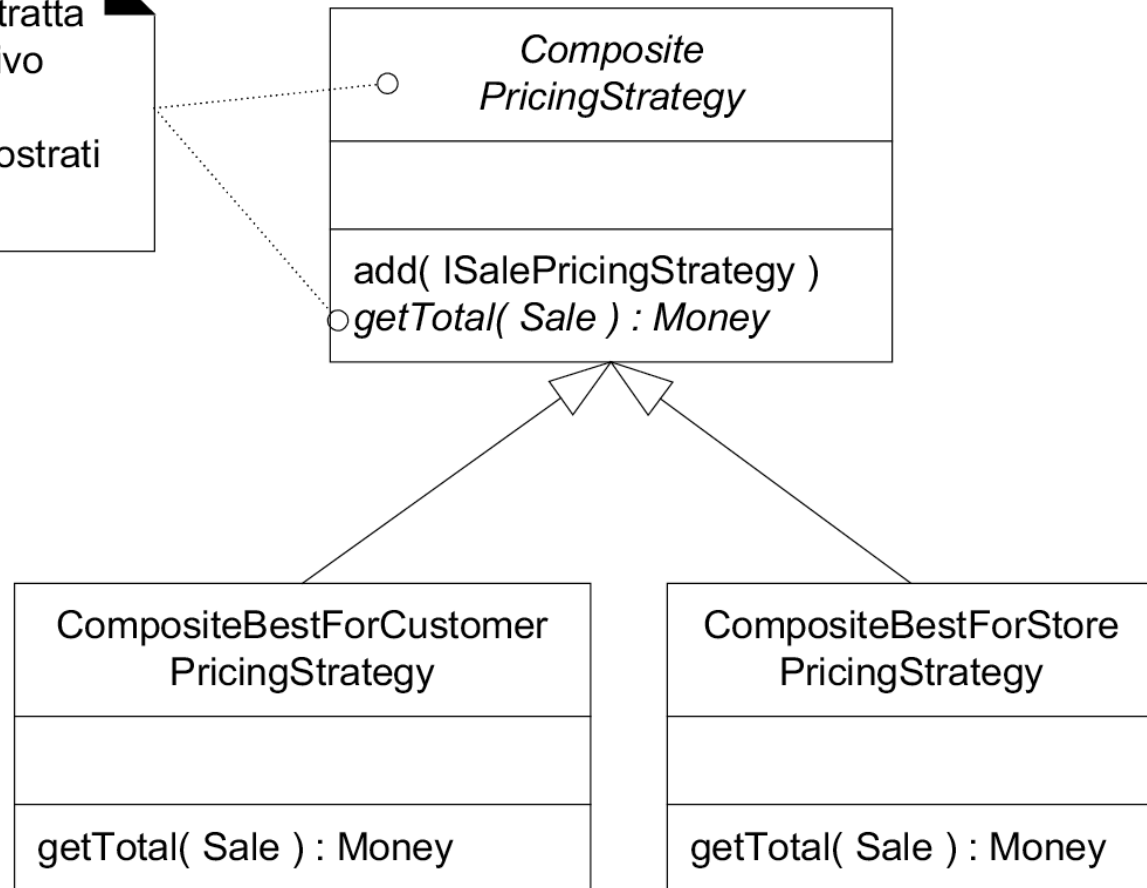
notazione UML: ISalePricingStrategy è un'interfaccia, non una classe;
questo è il modo che si usa in UML 2 per indicare un oggetto
la cui classe è sconosciuta, ma che implementa questa interfaccia



Superclassi astratte, metodi astratti ed eredità in UML

notazione UML: una classe astratta è mostrata con il nome in corsivo

anche i metodi astratti sono mostrati in corsivo



Implementazione

```
// superclasse tale che tutte le sue sottoclassi ereditano
// una lista di strategie

public abstract class CompositePricingStrategy
    implements ISalePricingStrategy {

    protected List<ISalePricingStrategy> strategies =
        new ArrayList<>();

    public add( ISalePricingStrategy s ) {
        strategies.add( s );
    }

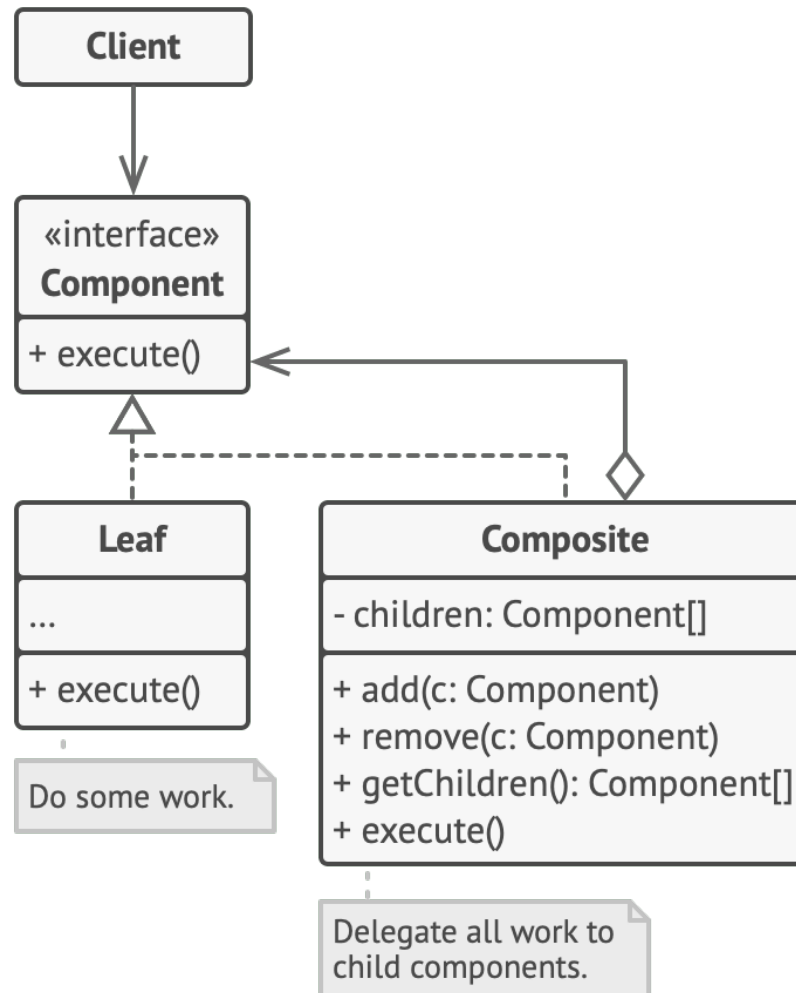
    public abstract Money getTotal( Sale sale );
} // fine della classe

// una Strategy Composite che restituisce il totale più basso
// delle sue SalePricingStrategy interne

public class CompositeBestForCustomerPricingStrategy
    extends CompositePricingStrategy {

    public Money getTotal( Sale sale ) {
        Money lowestTotal = new Money( Integer.MAX_VALUE );
        // itera su tutte le strategie interne
        for( ISalePricingStrategy strategy : strategies ) {
            Money total = strategy.getTotal( sale );
            lowestTotal = total.min( lowestTotal );
        }
        return lowestTotal;
    }
} // fine della classe
```

Struttura



Pattern Correlati

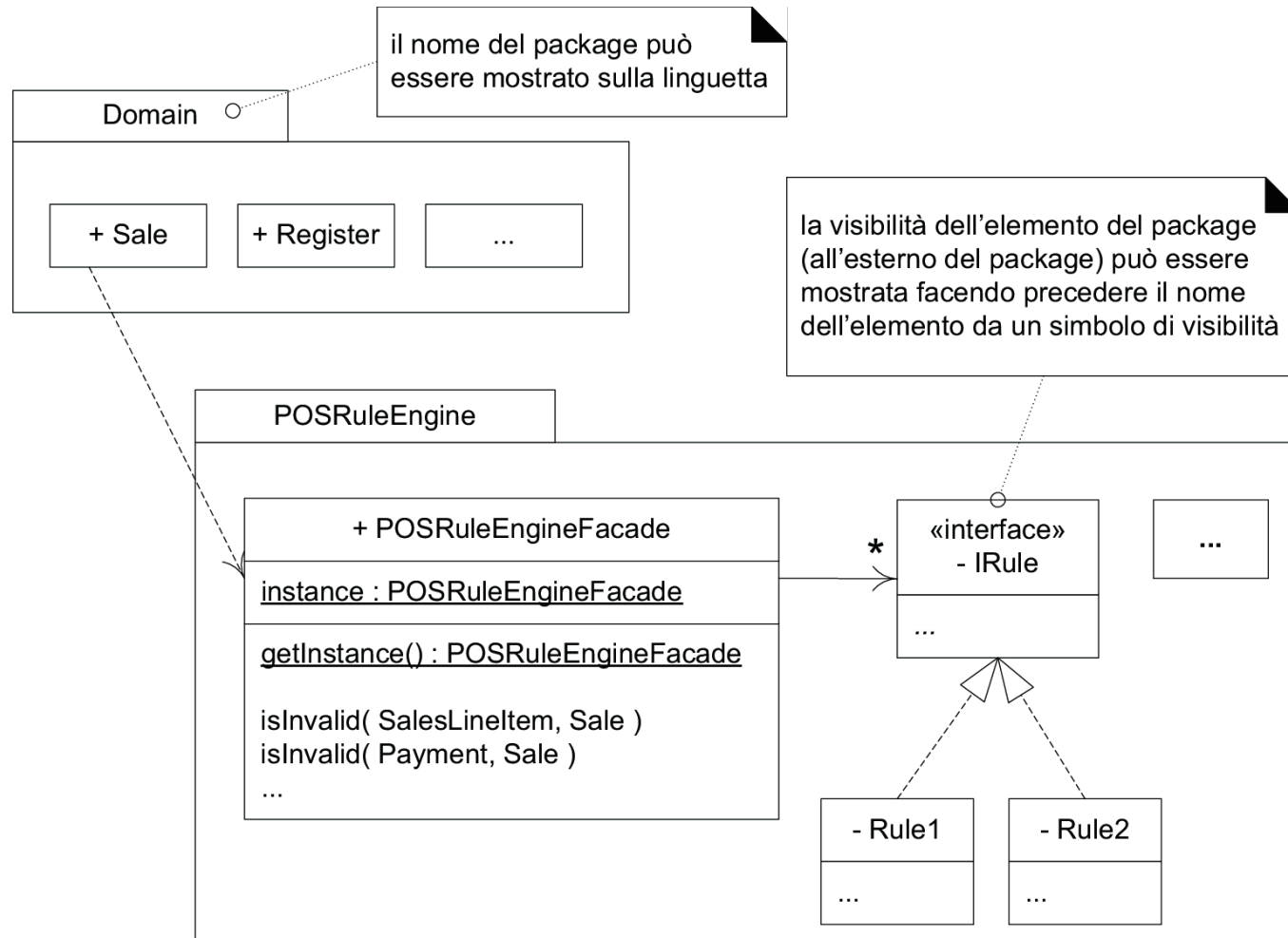
- Composite è spesso utilizzato con i pattern **Strategy** e **Command**
- Composite è basato su **Polymorphism** e fornisce **Protected Variations** su un client
 - *Cosicché questo non subisca effetti nel caso in cui suoi oggetti correlati siano atomici o composti*

Facade

Nome:	Facade
Problema:	È richiesta un'interfaccia comune e unificata per un insieme disparato di implementazioni o interfacce, come per definire un sottosistema. Può verificarsi un accoppiamento indesiderato a molti oggetti nel sottosistema, oppure l'implementazione del sottosistema può cambiare. Che cosa fare?
Soluzione: (consiglio)	Definisci un punto di contatto singolo con il sottosistema, ovvero un oggetto facade (facciata) che copre il sottosistema. Questo oggetto facade presenta un'interfaccia singola e unificata ed è responsabile della collaborazione con i componenti del sottosistema.

- Un Facade è un oggetto front end che rappresenta il punto di entrata singolo ai servizi di un sottosistema

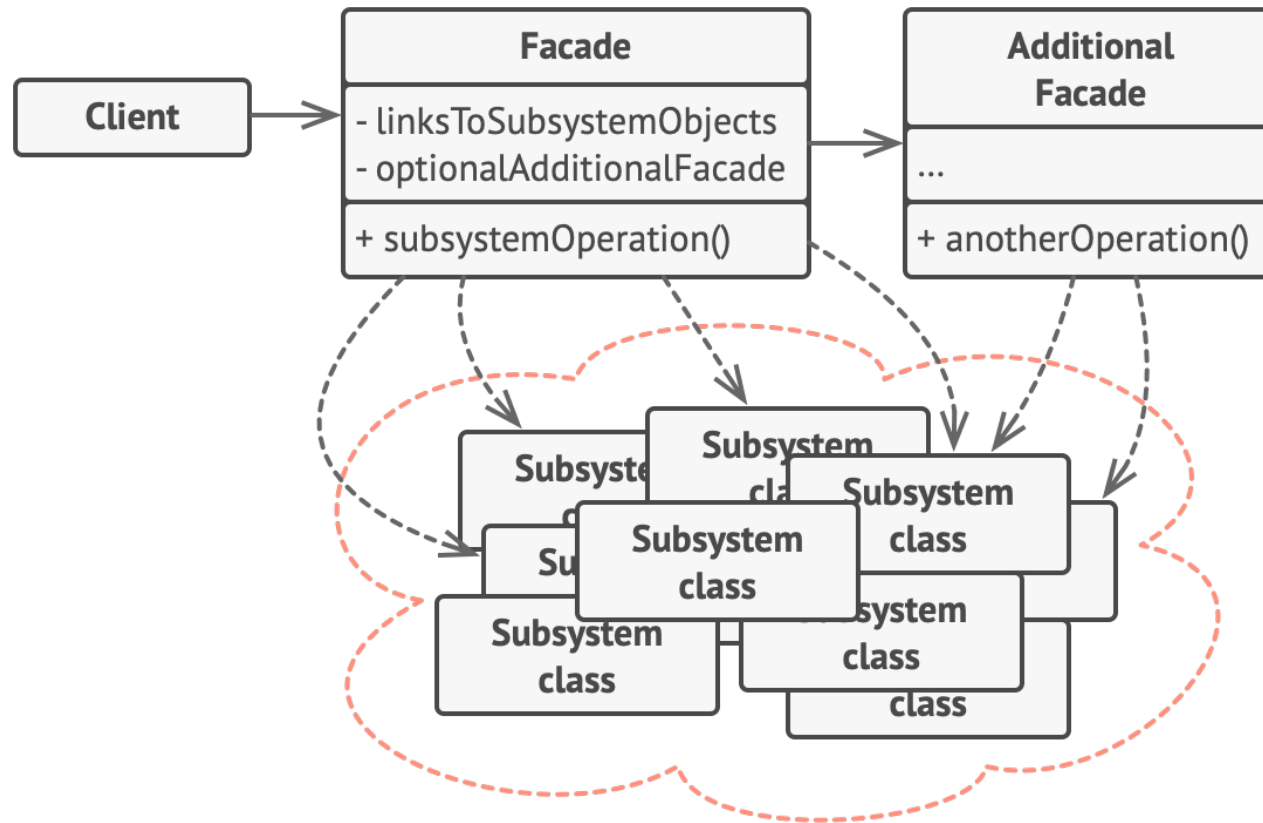
Diagramma dei Package UML con un Facade



Chiamata al Facade

```
public class Sale {  
  
    public void makeLineItem( ProductDescription desc, int quantity ) {  
        SalesLineItem sli = new SalesLineItem( desc, quantity );  
  
        // chiamata alla Facade  
        if ( POSRuleEngineFacade.getInstance().isInvalid(sli, this) ) {  
            return;  
        }  
        // se invece la regola è soddisfatta  
        lineItems.add( sli );  
    }  
  
    // ...  
} // fine della classe
```

Struttura



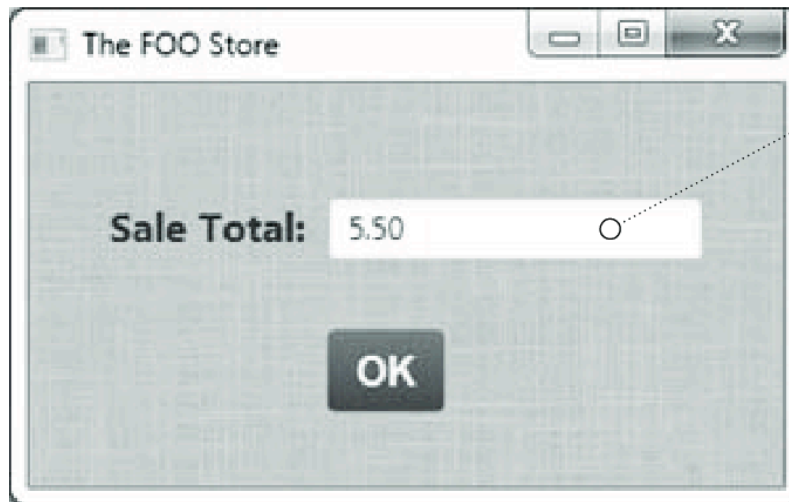
Pattern correlati

- Normalmente si accede a un Facade attraverso il pattern Singleton
- Un Facade fornisce Protected Variations dall'implementazione di un sottosistema, aggiungendo un oggetto Indirection che aiuta a sostenere Low Coupling
- Un facade controller è un tipo Facade
- Un oggetto Adapter può essere utilizzato per avvolgere l'accesso a sistemi esterni con interfacce variabili

Observer

Come si aggiunge la capacità per una finestra della GUI di aggiornare la visualizzazione del totale della vendita quando il totale cambia?

Obiettivo: quando cambia il totale della vendita, aggiornare la vista con il nuovo valore.



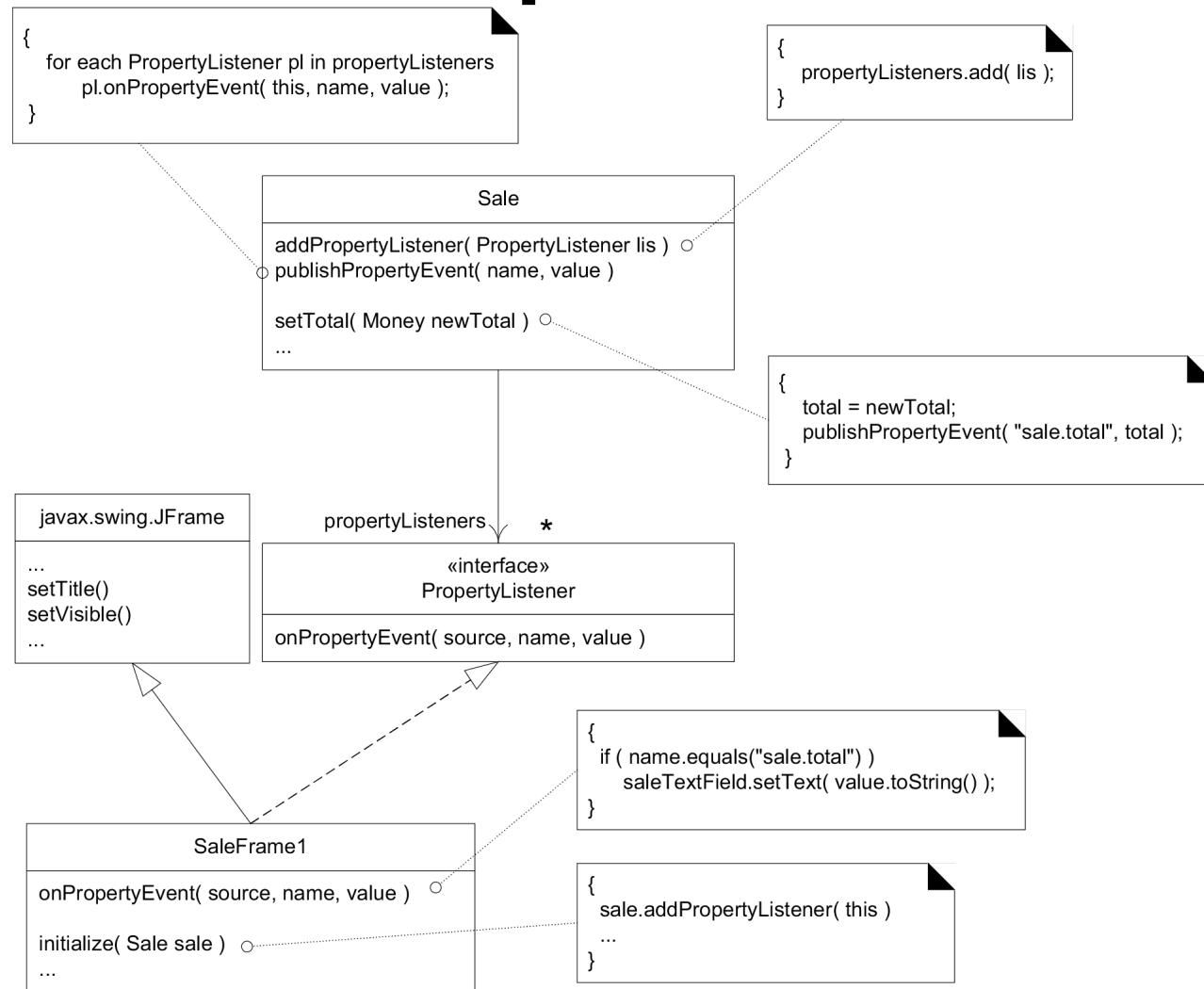
Observer

Nome:	Observer (Publish-Subscribe)
-------	-------------------------------------

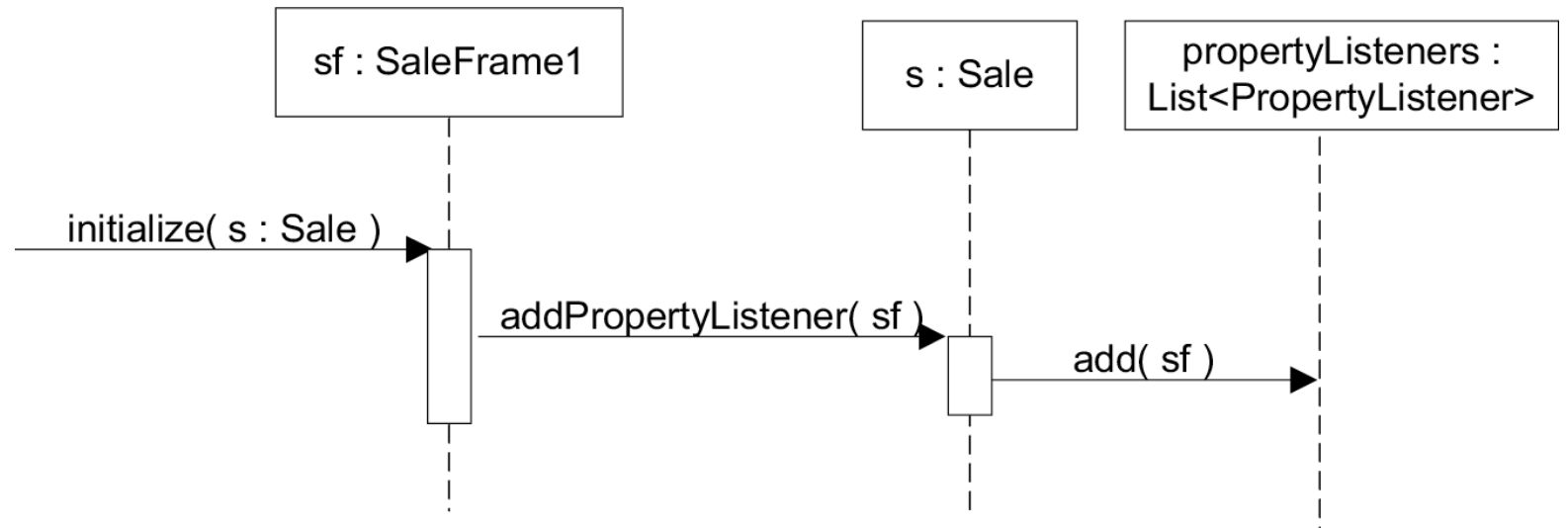
Problema:	Diversi tipi di oggetti <i>subscriber</i> (abbonato) sono interessati ai cambiamenti di stato o agli eventi di un oggetto <i>publisher</i> (editore). Ciascun subscriber vuole reagire in un suo modo proprio quando il publisher genera un evento. Inoltre, il publisher vuole mantenere un accoppiamento basso verso i subscriber. Che cosa fare?
-----------	---

Soluzione: (consiglio)	Definisci un'interfaccia <i>subscriber</i> o <i>listener</i> (ascoltatore). Gli oggetti subscriber implementano questa interfaccia. Il publisher registra dinamicamente i subscriber che sono interessati ai suoi eventi, e li avvisa quando si verifica un evento.
---------------------------	---

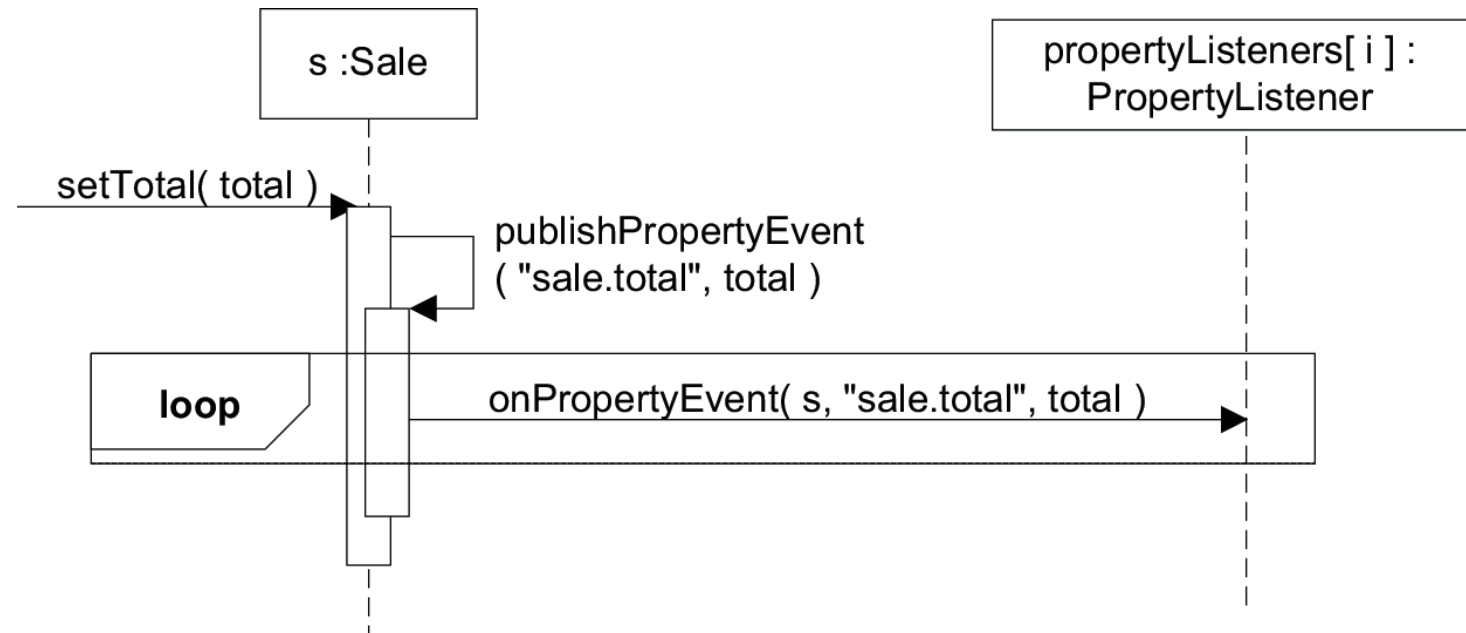
Applicazione del pattern Observer



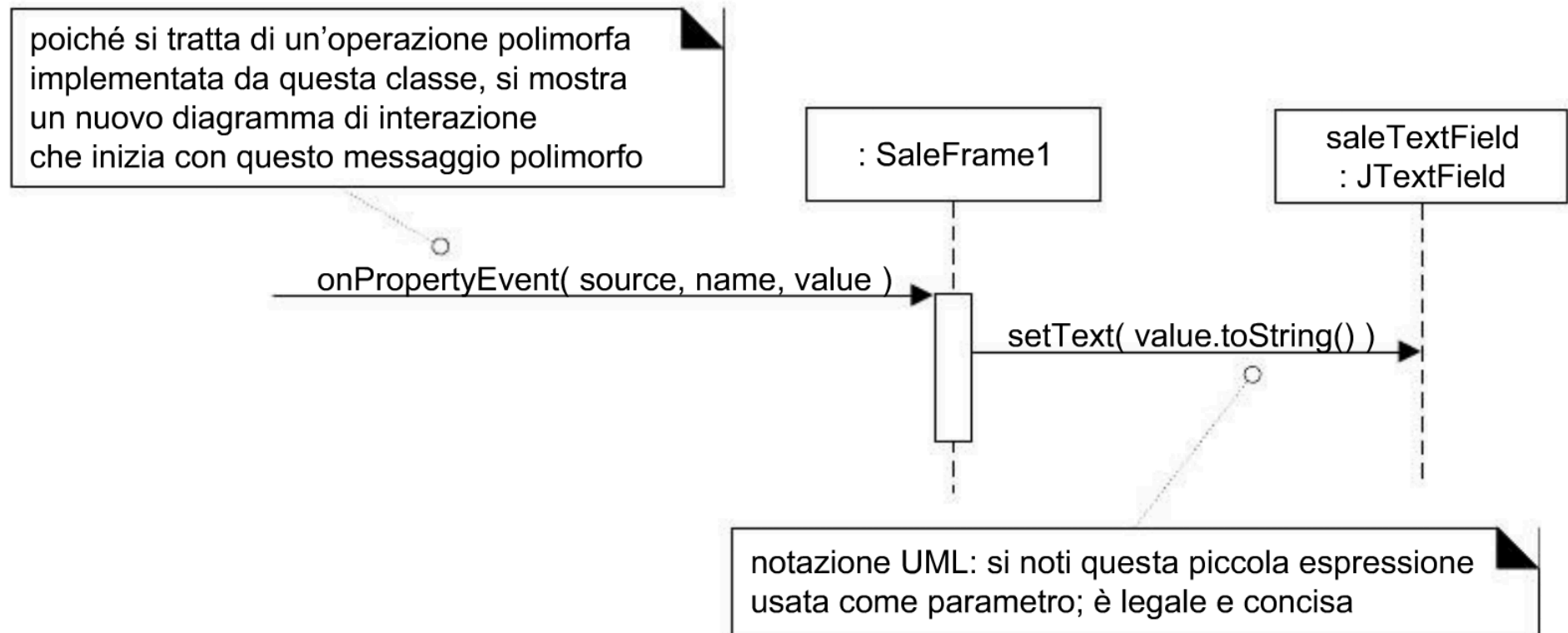
L'observer SaleFrame1 si abbona al Publisher Sale



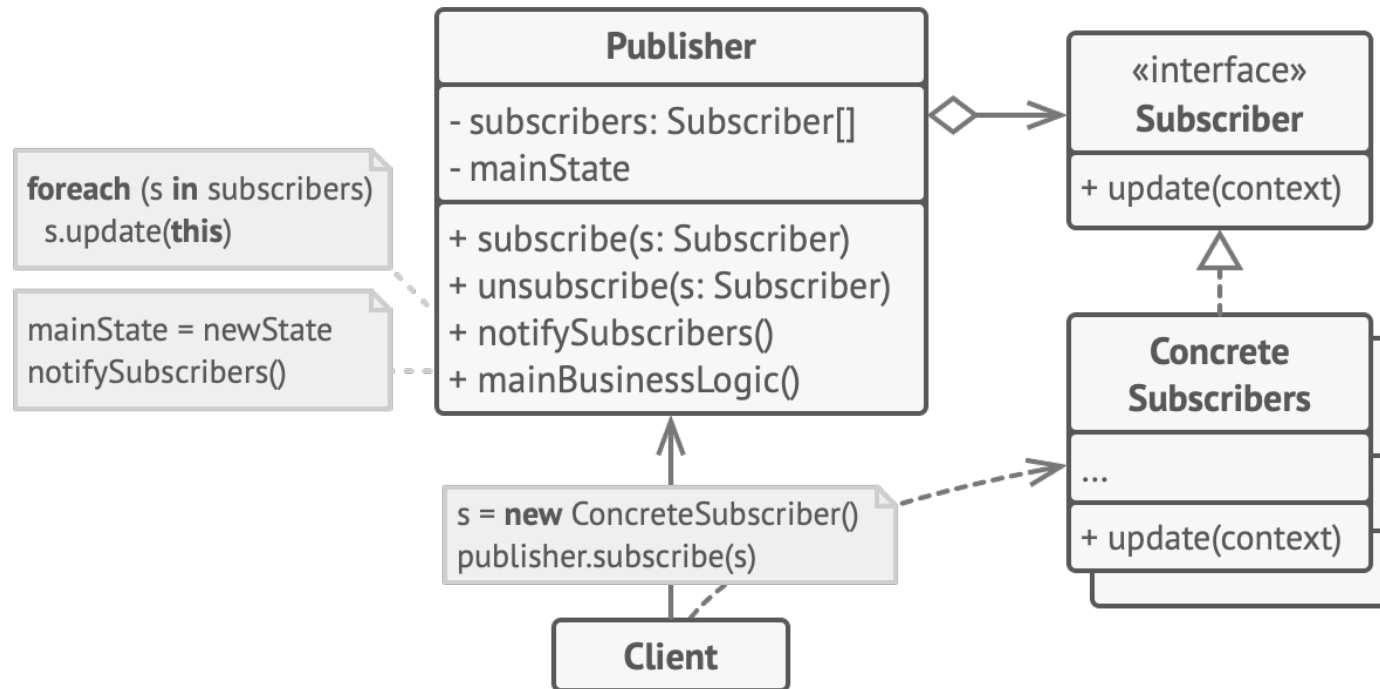
La Sale pubblica un evento proprietà a tutti i suoi subscriber



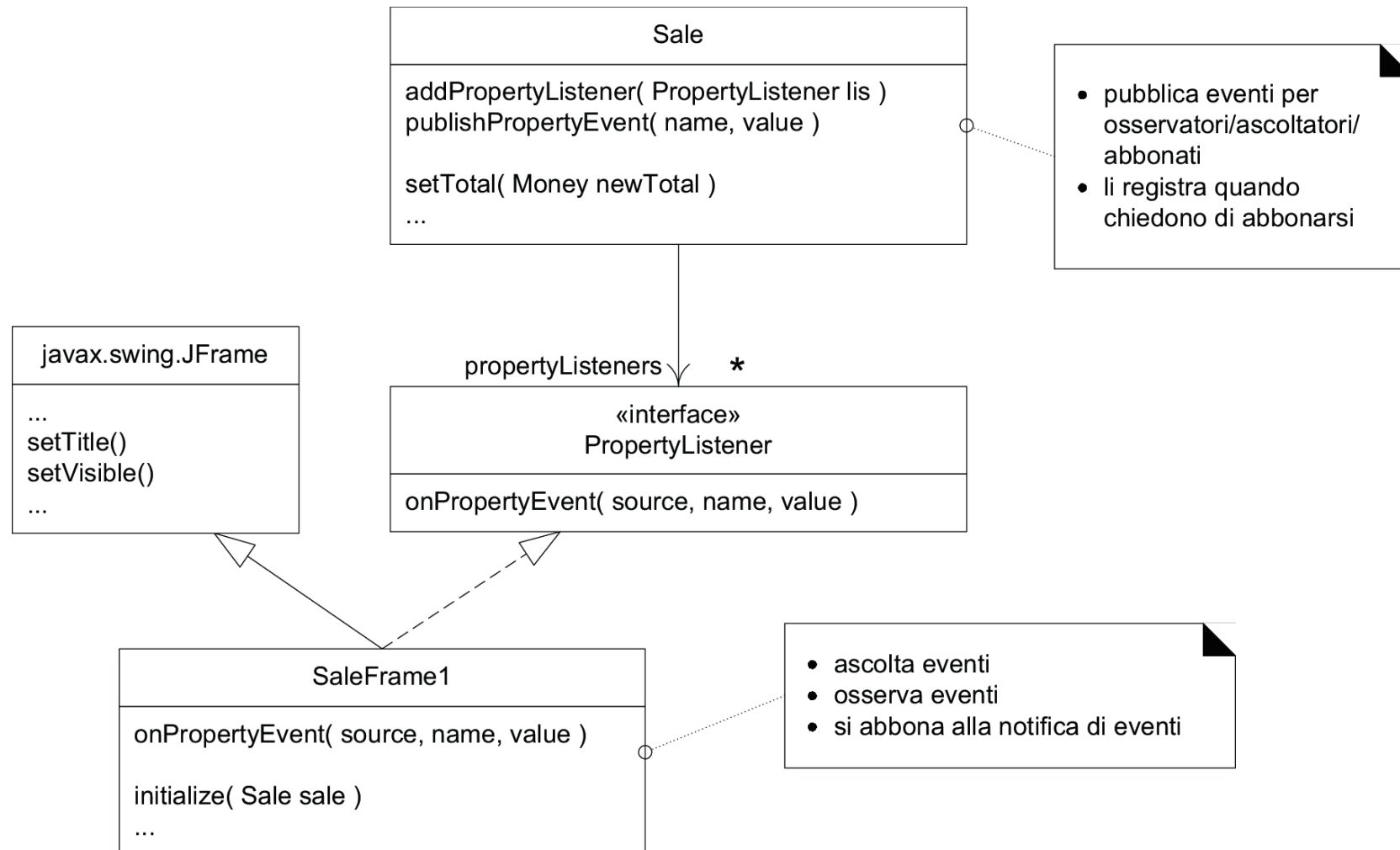
Visualizzazione del totale



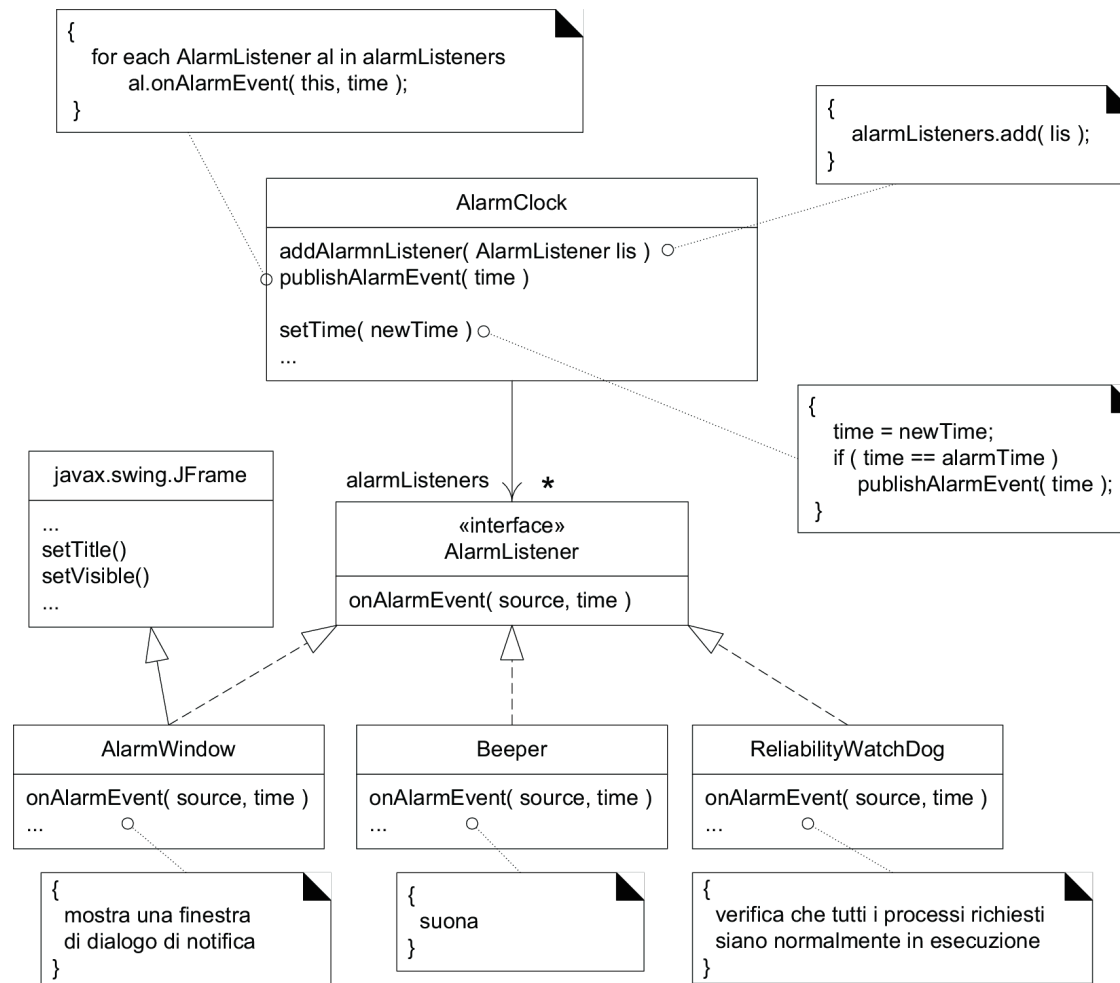
Struttura



Chi sono l'observer, il listener, il subscriber e il publisher?



Observer non serve solo a connettere le UI e gli oggetti modello



Riepilogo

Observer fornisce un modo per accoppiare in modo debole gli oggetti che devono comunicare

- I publisher conoscono i subscriber solo attraverso un'interfaccia
- I subscriber possono registrarsi (o cancellare la propria registrazione) dinamicamente con il publisher

Pattern correlati

- Observer è basato su **Polymorphism**
- Fornisce **Protected Variations** in termini di protezione del publisher dalla conoscenza della classe specifica degli oggetti con cui comunica quando genera un evento